МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное бюджетное образовательное учреждение высшего образования
«Российский экономический университет имени Г.В. Плеханова»

Пермский институт (филиал)

Отчет

по производственной практике

ПП.02.01 Технология разработки программного обеспечения
(индекс по РУП и наименование производственной практики)

Профессионального модуля ПМ.02 Осуществление интеграции программных модулей

Специальность 09.02.07 «Информационные системы и программирование»

Обучающийся _____ ____Садиуллина Алеся Дмитриевна_____
(подпись) (фамилия, имя, отчество)

Группы ____ИПо-41_____

Руководитель практической подготовки от профильной организации

_____Декан математического факультета ПГГПУ_____
(должность)

_____ _____Скорнякова Анна Юрьевна_____
(подпись) (фамилия, имя, отчество)

МП «16» марта 2026 года

Руководитель практической подготовки от института

_____ _____Берестов Дмитрий Борисович_____
(подпись) (фамилия, имя, отчество)

«16» марта 2026 года

Пермь, 2026 год

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ 3
РАЗДЕЛ 1. ОРГАНИЗАЦИОННО-ПОДГОТОВИТЕЛЬНЫЙ ЭТАП ПРОХОЖДЕНИЯ ПРАКТИКИ НА ПРЕДПРИЯТИИ 6
Тема 1.1. Инструктаж по прохождению производственной практики и правилам безопасности работы на предприятии 6
РАЗДЕЛ 2. ОЗНАКОМЛЕНИЕ СО СТРУКТУРОЙ ДЕЯТЕЛЬНОСТИ ПОДРАЗДЕЛЕНИЯ 9
Тема 2.1. Ознакомление с организацией работы на предприятии или в структурном подразделении 9
Тема 2.2. Ознакомление с должностными и функциональными обязанностями 11
РАЗДЕЛ 3. РАБОТА НА РАБОЧИХ МЕСТАХ ИЛИ В ПОДРАЗДЕЛЕНИЯХ ПРЕДПРИЯТИЯ 16
Тема 3.1. Ознакомление с организацией информационного обеспечения подразделения 16
Тема 3.2. Выполнение производственных заданий 19
3.2.1. использование выбранной системы контроля версий 19
3.2.2. разработка программного обеспечения 21
3.2.3. Использование методов для получения кода с заданной функциональностью и степенью качества 39
3.2.4. Инспектирование компонент программного обеспечения на предмет соответствия стандартам кодирования 41
3.2.5. Выполнение отладки программного модуля с использованием специализированных программных средств 43
3.2.6. Осуществление разработки тестовых наборов и тестовых сценариев для программного обеспечения 46
3.2.7. Интегрирование программных модулей 49
3.2.8. Верификация и аттестация программного обеспечения 51
Тема 3.3. Сбор и анализ материалов для оформления отчетной документации по практике 54
ЗАКЛЮЧЕНИЕ 56
СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ 60

ВВЕДЕНИЕ

Практика имеет целью комплексное освоение студентами всех видов профессиональной деятельности по специальности 09.02.07 «Информационные системы и программирование», формирование общих и профессиональных компетенций, а также приобретение необходимых умений и опыта практической работы по специальности.

Производственная практика направлена на освоение обучающимися профессиональных компетенций в процессе самостоятельного выполнения определенных видов работ, необходимых для осуществления профессиональной деятельности в максимально приближенных к производству условиях, в рамках модулей ППССЗ по основным видам профессиональной деятельности для освоения методов и приемов практического применения прикладных программных продуктов для программного обеспечения компьютерных систем.

В настоящей работе представлен отчет о прохождении производственной практики студента-практиканта. Основными видами деятельности которой являлись: разработка требований к программным модулям на основе анализа проектной и технической документации на предмет взаимодействия компонент; моделирование процесса разработки программного обеспечения; интеграция модулей в программное обеспечение; отладка программного модуля с использованием специализированных программных средств; разработка тестовых наборов и тестовых сценариев для программного обеспечения.

Местом прохождения производственной практики является Федеральное государственное бюджетное образовательное учреждение высшего образования «Пермский государственный гуманитарно-педагогический университет» (ПГГПУ).

Структурным подразделением, в котором непосредственно проходила практика, является Математический факультет. Деканом математического

факультета является кандидат педагогических наук, доцент Скорнякова Анна Юрьевна.

Основными целями деятельности математического факультета, согласно Положению о факультете, является обеспечение эффективного управления деятельностью кафедр и иных структурных подразделений, реализация учебной, методической, научно-исследовательской деятельности по направлениям подготовки «Педагогическое образование» (профили «Математика» и «Информатика»). В состав факультета входят кафедры, деканат, компьютерные классы и учебные лаборатории, оснащенные современной вычислительной техникой, что позволяет осуществлять образовательный процесс на высоком технологическом уровне и служит материально-технической базой для прохождения практики.

Целью производственной практики является закрепление и углубление теоретических знаний, полученных в процессе обучения, и приобретение практических навыков в области разработки, интеграции, отладки и тестирования программных модулей для автоматизации деятельности предприятий.

Задачи практики (освоение обучающимися профессиональных и общих компетенций):

1. Выполнять интеграцию модулей в программное обеспечение;
2. Выполнять отладку программного модуля с использованием специализированных программных средств;
3. Осуществлять разработку тестовых наборов и тестовых сценариев для программного обеспечения;
4. Производить инспектирование компонент программного обеспечения на предмет соответствия стандартам кодирования;
5. Выбирать способы решения задач профессиональной деятельности, применительно к различным контекстам;
6. Осуществлять поиск, анализ и интерпретацию информации, необходимой для выполнения задач профессиональной деятельности

7. Планировать и реализовывать собственное профессиональное и личностное развитие;

8. Работать в коллективе и команде, эффективно взаимодействовать с коллегами, руководством, клиентами

9. Осуществлять устную и письменную коммуникацию на государственном языке Российской Федерации с учетом особенностей социального и культурного контекста.

10. Использовать информационные технологии в профессиональной деятельности

11. Пользоваться профессиональной документацией на государственном и иностранном языках

РАЗДЕЛ 1. ОРГАНИЗАЦИОННО-ПОДГОТОВИТЕЛЬНЫЙ ЭТАП ПРОХОЖДЕНИЯ ПРАКТИКИ НА ПРЕДПРИЯТИИ

Тема 1.1. Инструктаж по прохождению производственной практики и правилам безопасности работы на предприятии

Прохождение производственной практики началось с организационно-подготовительных мероприятий, предусмотренных программой. Прибыв на место практики в ПГГПУ, был пройден вводный инструктаж, на котором руководитель практики ознакомил с целями, задачами и календарным планом практики, а также с правилами внутреннего трудового распорядка университета.

Ключевым этапом стало прохождение первичного инструктажа по охране труда и технике безопасности на рабочем месте. В ходе инструктажа были подробно рассмотрены следующие аспекты:

- Общие правила техники безопасности: запрещается приступать к работе без знания инструкций по эксплуатации оборудования, использовать неисправные электрические приборы и оргтехнику, загромождать рабочее место и проходы к средствам пожаротушения.
- Правила работы на рабочем месте с ПЭВМ: организация рабочего места с соблюдением эргономических требований (расстояние до монитора не менее 50 см, правильная посадка), режим труда и отдыха для снижения зрительного и мышечного напряжения.
- Правила работы с электрическими приборами: необходимость визуального осмотра состояния электропроводки и шнуров питания перед началом работы, запрет на оставление работающего оборудования без присмотра и самостоятельное вскрытие аппаратуры.
- Правила пожарной безопасности: изучены расположение плана эвакуации, места нахождения огнетушителей и аптечки первой помощи. Усвоен порядок действий при возникновении пожара или задымления.

Факт проведения и усвоения материала инструктажа был зафиксирован в соответствующем журнале установленного образца. После инструктажа был произведен допуск к работе на закрепленном за мной рабочем месте.

Базой практики является математический факультет ФГБОУ ВО «Пермский государственный гуманитарно-педагогический университет» (ПГГПУ). Деканом математического факультета является кандидат педагогических наук, доцент Скорнякова Анна Юрьевна.

Основными целями деятельности математического факультета, согласно Положению о факультете, является обеспечение эффективного управления деятельностью кафедр, реализация учебной, методической и научно-исследовательской деятельности по направлениям подготовки «Педагогическое образование» (профили «Математика» и «Информатика»). В состав факультета входят кафедры, деканат, компьютерные классы и учебные лаборатории, оснащенные современной вычислительной техникой. Непосредственным местом прохождения практики стала кафедра информатики и вычислительной техники.

Рабочее место, закрепленное на время практики, расположено в компьютерном классе, используемом для учебных занятий и самостоятельной работы. Оно представляет собой автоматизированное рабочее место (АРМ) на базе персонального компьютера.

Аппаратное обеспечение рабочего места:

- Процессор: Intel Core i5 с тактовой частотой 3.20 GHz.
- Оперативная память: 16 ГБ типа DDR4.
- Накопитель: твердотельный накопитель (SSD) объемом 512 ГБ.
- Монитор: ЖК-монитор с диагональю 23.8 дюйма и разрешением Full HD (1920x1080).
- Периферийные устройства: клавиатура, оптическая мышь, многофункциональное устройство (МФУ).

Программное обеспечение рабочего места:

На компьютере установлена лицензионная операционная система Microsoft Windows 10 Pro. Для выполнения поставленных задач и обеспечения учебного процесса на ПК установлен следующий набор прикладного программного обеспечения:

- Офисный пакет: Microsoft Office Standard 2019/2021 (Word, Excel, PowerPoint, Outlook).
- Интернет-браузеры: Google Chrome, Yandex Browser.
- Антивирусное ПО: Kaspersky Endpoint Security.
- Средства разработки и администрирования: В рамках выполнения индивидуального задания мною были дополнительно установлены Microsoft Visual Studio 2022 Community Edition, Visual Studio Code, СУБД PostgreSQL и MySQL Community Edition, а также инструменты для работы с Git.

Данная конфигурация оборудования и ПО полностью соответствовала требованиям для выполнения индивидуального задания по разработке программного продукта.

РАЗДЕЛ 2. ОЗНАКОМЛЕНИЕ СО СТРУКТУРОЙ ДЕЯТЕЛЬНОСТИ ПОДРАЗДЕЛЕНИЯ

Тема 2.1. Ознакомление с организацией работы на предприятии или в структурном подразделении

Производственная практика проходила на базе математического факультета федерального государственного бюджетного образовательного учреждения высшего образования «Пермский государственный гуманитарно-педагогический университет» (ПГГПУ). Данное образовательное учреждение является крупным вузом Пермского края, осуществляющим подготовку высококвалифицированных педагогических кадров для региона и других субъектов Российской Федерации. Математический факультет, на базе которого осуществлялось прохождение практики, выступает одним из ключевых структурных подразделений университета.

Согласно «Положению о математическом факультете» (СТО ПСП 12-08-02-2020/01), факультет создан для реализации образовательных программ высшего образования по направлениям подготовки, связанным с педагогическим образованием, в частности, для подготовки учителей математики и информатики. Деканом математического факультета является кандидат педагогических наук, доцент Скорнякова Анна Юрьевна.

В ходе детального изучения структуры факультета было установлено, что она представляет собой сложную иерархическую систему, включающую в себя несколько взаимосвязанных элементов, каждый из которых выполняет строго определенные функции. Структурная организация математического факультета включает следующие подразделения:

Деканат представляет собой административно-управленческий аппарат факультета, который осуществляет оперативное управление, контроль учебного процесса, работу с контингентом студентов (переводы, отчисления, восстановления), подготовку расписания учебных занятий, организацию зачетно-экзаменационных сессий, стипендиальное обеспечение и

координацию работы всех нижестоящих подразделений. В состав деканата входят заместители декана по учебной работе, научной работе и воспитательной работе, а также документовед (секретарь) и специалисты по учебно-методической работе.

Кафедры представляют собой основные учебно-научные структурные подразделения факультета, отвечающие за непосредственную реализацию образовательного процесса. В составе математического факультета функционируют две кафедры:

- кафедра высшей математики и методики обучения математике;
- кафедра информатики и вычислительной техники (ВТ).

Кафедра информатики и ВТ занимается проведением учебных занятий по дисциплинам информационного профиля, разработкой рабочих программ дисциплин (РПД) и фондов оценочных средств, организацией научно-исследовательской работы студентов и преподавателей в области современных информационных технологий.

Учебно-вспомогательные и научные подразделения включают в себя компьютерные классы, учебно-методический кабинет, научно-методическую лабораторию «Методическое сопровождение обучения математике в основной общеобразовательной школе», кабинет истории математики и математического образования. Данные подразделения обеспечивают материально-техническую базу для учебного процесса, хранение методических материалов и проведение научных исследований.

Взаимодействие внутри факультета построено по классическому линейно-функциональному принципу, который обеспечивает четкое распределение полномочий и ответственности между различными уровнями управления. Декан факультета осуществляет общее руководство работой кафедр и деканата через систему распоряжений и приказов. Заведующие кафедрами, в свою очередь, управляют профессорско-преподавательским составом, распределяют учебную нагрузку и контролируют методическую работу. Учебно-вспомогательный персонал, включая лаборантов, инженеров

и специалистов, обеспечивает техническую и организационную поддержку всех процессов, взаимодействуя как с профессорско-преподавательским составом, так и с деканатом.

В ходе ознакомления с организацией работы был детально рассмотрен режим работы сотрудников факультета. Для учебно-вспомогательного персонала установлен нормированный рабочий день продолжительностью 40 часов в неделю, строго регламентированный Правилами внутреннего трудового распорядка ПГГПУ. Для профессорско-преподавательского состава характерен более гибкий график работы, который определяется расписанием учебных занятий, составляемым и контролируемым деканатом.

На основе всей собранной информации была составлена характеристика предприятия. ПГГПУ представляет собой государственное образовательное учреждение высшего образования, финансируемое из федерального бюджета. Его основной целью является удовлетворение потребностей общества и государства в квалифицированных педагогических кадрах, способных осуществлять профессиональную деятельность в условиях современной образовательной среды, активно использующей информационно-коммуникационные технологии. Математический факультет играет ключевую роль в этом процессе, обеспечивая высокий уровень фундаментальной математической и методической подготовки будущих учителей, а также их компетентность в области информатики и вычислительной техники.

Тема 2.2. Ознакомление с должностными и функциональными обязанностями

Одним из важнейших этапов производственной практики стало детальное изучение должностной инструкции сотрудника, непосредственно отвечающего за поддержку программно-аппаратного комплекса кафедры, а также за документационное обеспечение ее деятельности. Мною была проанализирована должностная инструкция лаборанта кафедры информатики и вычислительной техники (автор-составитель: Отинова А.А., утвержденная в

установленном порядке). Данный документ является локальным нормативным актом, определяющим трудовые функции, права, обязанности и ответственность работника, занимающего данную должность.

Согласно изученному документу, лаборант относится к категории учебно-вспомогательного персонала и подчиняется непосредственно заведующему кафедрой. На должность лаборанта назначается лицо, имеющее среднее профессиональное образование без предъявления требований к стажу работы, либо лицо, получающее высшее образование по соответствующему направлению подготовки, что делает эту позицию доступной для студентов старших курсов.

В своей профессиональной деятельности лаборант кафедры руководствуется Трудовым кодексом Российской Федерации, Федеральным законом «Об образовании в Российской Федерации», Уставом ПГГПУ, приказами ректора университета, распоряжениями декана факультета и непосредственного руководителя - заведующего кафедрой, а также правилами внутреннего трудового распорядка и иными локальными нормативными актами образовательной организации.

В ходе детального анализа должностной инструкции были выделены и систематизированы все функциональные обязанности данного специалиста, которые можно разделить на несколько ключевых блоков:

Документационное обеспечение деятельности кафедры. Данная функция является одной из основных в работе лаборанта. Она включает в себя печать и копирование учебно-методических материалов, разрабатываемых профессорско-преподавательским составом (рабочих программ дисциплин, фондов оценочных средств, методических указаний для студентов, учебных пособий). Лаборант осуществляет оформление планов научно-исследовательской и методической работы кафедры, обеспечивает техническое сопровождение заседаний кафедры (подготовка повесток, раздаточных материалов) и ведение их протоколов. В обязанности также входит регистрация и учет курсовых и выпускных квалификационных работ

обучающихся, включая проверку соответствия оформления установленным требованиям. Лаборант обеспечивает вывод на печать индивидуальных планов профессорско-преподавательского состава из автоматизированной системы «Нагрузка» и организует сбор необходимых подписей.

Организация делопроизводства и архива. Лаборант ведет делопроизводство кафедры в строгом соответствии с утвержденной номенклатурой дел, что предполагает правильное формирование дел, их оформление, обеспечение сохранности документов и подготовку к последующей передаче в архив университета по истечении установленных сроков хранения. Данная функция требует внимательности, аккуратности и знания основ архивного дела.

Информационно-техническое и материальное обеспечение. В обязанности лаборанта входит ведение учета имеющегося на кафедре оборудования, оргтехники, расходных материалов и учебно-методических пособий. Он оформляет заявки на приобретение нового оборудования или ремонт существующего, взаимодействуя с соответствующими административно-хозяйственными службами университета. Особое внимание в инструкции уделено функциям, связанным с обслуживанием лабораторного оборудования. Лаборант обязан следить за его сохранностью и исправным состоянием, осуществлять его наладку и первичную настройку, подготавливать приборы к проведению лабораторных работ и экспериментов, участвовать в выполнении экспериментов, снимать показания приборов и обрабатывать полученные результаты. В рамках выполнения данной функции лаборант также обеспечивает техническую поддержку при проведении учебных занятий в компьютерных классах.

Работа с персональными данными. Пункт 3.14 должностной инструкции прямо предписывает лаборанту при выполнении своих служебных обязанностей обрабатывать персональные данные преподавателей и обучающихся, сохраняя их конфиденциальность и не допуская разглашения. Данное требование подчеркивает важность соблюдения законодательства о

защите персональных данных (Федеральный закон № 152-ФЗ «О персональных данных») и требует от сотрудника соответствующего уровня ответственности.

Выполнение поручений и соблюдение дисциплины. Лаборант обязан выполнять иные поручения непосредственного руководителя (заведующего кафедрой), связанные с обеспечением деятельности кафедры, соблюдать трудовую дисциплину, правила охраны труда, техники безопасности и пожарной безопасности. Также в его обязанности входит ежемесячное представление в бухгалтерию табеля учета рабочего времени сотрудников кафедры.

Анализ должностной инструкции позволил также изучить права лаборанта. Он имеет право знакомиться с проектами решений руководства, касающихся его деятельности, вносить на рассмотрение заведующего кафедрой предложения по совершенствованию работы, связанной с предусмотренными данной инструкцией обязанностями, запрашивать лично или по поручению руководителя информацию и документы, необходимые для выполнения его должностных обязанностей.

Ответственность лаборант несет за неисполнение или ненадлежащее исполнение своих обязанностей, предусмотренных настоящей инструкцией, в пределах, определенных трудовым законодательством Российской Федерации; за правонарушения, совершенные в процессе осуществления своей деятельности, в пределах, определенных административным, уголовным и гражданским законодательством; за нарушение Устава университета, правил внутреннего трудового распорядка, правил и норм охраны труда и техники безопасности, а также за причинение материального ущерба университету.

Детальное изучение должностной инструкции лаборанта позволило сформировать четкое представление о структуре его трудовых функций, правах и обязанностях, а также о его роли в обеспечении бесперебойного функционирования кафедры. Полученные знания были использованы при

планировании и выполнении индивидуального задания, основной упор в котором был сделан на практические работы по администрированию программного обеспечения, работе с базами данных и обеспечению информационной безопасности, что полностью соответствует профессиональному модулю ПМ.02 «Осуществление интеграции программных модулей» и профилю подготовки специалиста в области информационных систем и программирования.

РАЗДЕЛ 3. РАБОТА НА РАБОЧИХ МЕСТАХ ИЛИ В ПОДРАЗДЕЛЕНИЯХ ПРЕДПРИЯТИЯ

Тема 3.1. Ознакомление с организацией информационного обеспечения подразделения

Информационное обеспечение деятельности математического факультета и кафедры информатики и вычислительной техники базируется на использовании нескольких взаимосвязанных информационных систем и технологий, которые были изучены в ходе прохождения практики. Информационная инфраструктура факультета представляет собой комплекс аппаратных и программных средств, предназначенных для обеспечения учебного процесса, научной деятельности и административной работы.

В ходе ознакомления были проанализированы информационные системы и программные средства, задействованные в повседневной деятельности подразделения:

Автоматизированная система «Нагрузка» используется на уровне университета для планирования и учета учебной нагрузки профессорско-преподавательского состава. Данная система позволяет автоматизировать процесс распределения учебных поручений, расчета часов по дисциплинам и формирования итоговой отчетности по нагрузке кафедры. Лаборант кафедры непосредственно взаимодействует с данной системой: после утверждения нагрузки на учебный год осуществляется вывод на печать индивидуальных планов преподавателей, организуется их ознакомление и сбор подписей.

Корпоративная информационная система управления вузом обеспечивает комплексную автоматизацию процессов управления, включая внутренний документооборот, бухгалтерский учет, кадровый учет и управление контингентом студентов. Доступ к определенным модулям этой системы имеют сотрудники деканата и руководители подразделений. Система построена на принципах единой базы данных, что исключает дублирование информации и обеспечивает ее целостность.

Официальный сайт ПГГПУ (pspu.ru) служит основным инструментом для внешнего информационного взаимодействия. На сайте размещается официальная информация о деятельности вуза, факультетов и кафедр, сведения о приемной кампании, новости и объявления. На странице кафедры информатики и ВТ представлена информация о профессорско-преподавательском составе, направлениях подготовки, учебных планах, научных мероприятиях и достижениях студентов. Лаборант может участвовать в подготовке информации для обновления раздела кафедры на сайте.

Электронная информационно-образовательная среда (ЭИОС) является ключевым элементом информационного обеспечения образовательного процесса. ЭИОС предоставляет студентам и преподавателям круглосуточный удаленный доступ к учебным материалам (лекциям, методическим указаниям, практикумам), электронной библиотеке (ЭБС), системе дистанционного обучения (СДО) на платформе Moodle, а также к сервисам для проверки уникальности текстов и организации вебинаров. Доступ к ЭИОС осуществляется через личные кабинеты студентов и преподавателей с использованием индивидуальных логинов и паролей.

В ходе знакомства с процессами сбора, регистрации и обработки информации было установлено, что они строго регламентированы на всех уровнях. Данные о текущей успеваемости и посещаемости студентов собираются преподавателями в электронные журналы, которые ведутся в рамках ЭИОС. Эти данные затем агрегируются в деканате, где на их основе формируются ведомости промежуточной аттестации, рассчитывается академический рейтинг студентов и готовятся проекты приказов о назначении государственной академической стипендии. Информация о движении контингента (зачисления, переводы, отчисления, восстановления) оперативно вносится в единую базу данных и служит основой для подготовки статистической отчетности по формам ВПО-1 и другим.

Особое внимание при изучении информационного обеспечения было уделено применяемым методам защиты информации. В ПГГПУ реализована комплексная многоуровневая система информационной безопасности, соответствующая современным требованиям и включающая следующие меры:

Организационные меры представлены разработанными и внедренными внутренними инструкциями, регламентами и политиками, регламентирующими порядок работы с информацией и персональными данными. Все сотрудники, включая учебно-вспомогательный персонал, при приеме на работу предупреждаются об ответственности за разглашение персональных данных и подписывают соответствующие обязательства (как прямо указано в должностной инструкции лаборанта).

Программно-технические меры включают:

- Антивирусную защиту: на всех компьютерах, включая рабочие места в компьютерных классах, установлено лицензионное антивирусное программное обеспечение Kaspersky Endpoint Security с централизованным управлением и регулярным автоматическим обновлением баз.
- Разграничение доступа: внедрена система разграничения прав доступа пользователей к ресурсам локальной сети и корпоративным информационным системам. Доступ осуществляется на основе индивидуальных логинов и паролей, при этом политика безопасности требует использования сложных паролей и их периодической смены.
- Сетевую защиту: на периметре сети университета используются межсетевые экраны (файрволы) для фильтрации трафика и предотвращения несанкционированного проникновения извне. Также применяются системы обнаружения вторжений.

Правовые меры обеспечиваются тем, что вся деятельность в области информационной безопасности строится в строгом соответствии с Конституцией РФ, Федеральным законом «Об информации, информационных технологиях и о защите информации», Федеральным законом «О

персональных данных», а также локальными нормативными актами университета.

Таким образом, организация информационного обеспечения подразделения соответствует современным требованиям и создает необходимые условия как для образовательного процесса, так и для выполнения задач по разработке программного обеспечения в рамках производственной практики.

Тема 3.2. Выполнение производственных заданий

В рамках выполнения производственных заданий была разработана подсистема учета партнеров для компании «Мастер пол» согласно техническому заданию. Весь процесс разработки и сопутствующие работы описаны ниже.

3.2.1. использование выбранной системы контроля версий

В процессе разработки программного обеспечения для управления версиями исходного кода, отслеживания изменений и обеспечения возможности отката к предыдущим состояниям использовалась распределенная система контроля версий Git. Выбор данной системы обоснован ее повсеместным распространением в индустрии разработки программного обеспечения, высокой скоростью работы, надежностью и мощными возможностями для ветвления и слияния кода. Git является стандартом де-факто для современных проектов по разработке ПО.

В качестве удаленного репозитория для хранения кода и обеспечения доступа к нему для проверки руководителем практики была выбрана платформа GitHub. Данная платформа предоставляет удобный веб-интерфейс для просмотра кода, управления версиями и совместной работы. Работа с Git велась как через командную строку, так и с использованием графического интерфейса, встроенного в среду разработки Visual Studio 2022, что позволило освоить оба подхода к управлению версиями.

Основные этапы работы с Git в ходе выполнения производственных заданий включали:

1. Инициализация репозитория. Локальный репозиторий был создан в корневой папке с проектом. Для этого использовалась команда git init, которая создает скрытую директорию .git, содержащую всю служебную информацию системы контроля версий.

2. Создание коммитов. После реализации каждой значимой функциональности (например, создание моделей данных, добавление ViewModel, разработка окна редактирования, реализация алгоритма расчета скидок) изменения фиксировались с помощью команд git add и git commit. Команда git add использовалась для добавления измененных файлов в область подготовленных изменений (staging area), после чего команда git commit создавала коммит с зафиксированным сообщением. Комментарии к коммитам составлялись на русском языке, кратко и информативно описывая суть внесенных изменений (например, "feat: добавлен DiscountCalculator", "fix: исправлена загрузка типов партнеров", "docs: обновлена диаграмма классов").

3. Подключение удаленного репозитория. Локальный репозиторий был связан с удаленным репозиторием на GitHub с помощью команды git remote add origin <URL-репозитория>. Данное действие позволяет синхронизировать локальные изменения с облачным хранилищем.

4. Отправка изменений. По завершении рабочего сеанса или после реализации законченного блока функциональности все изменения отправлялись в удаленный репозиторий командой git push origin main. Это обеспечило сохранность кода, возможность его проверки руководителем практики в любой момент, а также создание резервной копии проекта.

Использование системы контроля версий позволило поддерживать структурированную историю разработки, гарантировало сохранность всех промежуточных и финальных версий программного продукта, а также способствовало формированию профессиональных навыков работы с инструментарием современной разработки.

3.2.2. разработка программного обеспечения

1. Выбор модели разработки

Для выполнения поставленной задачи по созданию подсистемы учета партнеров была выбрана итерационная модель разработки. Данный выбор обусловлен спецификой проекта и требованиями к процессу разработки. Итерационная модель предполагает разбиение процесса создания программного продукта на последовательность итераций, каждая из которых представляет собой законченный цикл разработки, приводящий к созданию работоспособного прототипа с новой функциональностью.

Обоснование выбора итерационной модели:

- Требования к подсистеме были четко определены изначально (просмотр списка партнеров, добавление и редактирование данных, просмотр истории продаж, расчет скидок), что позволило спланировать итерации в соответствии с этими функциональными блоками.
- Возможность последовательной разработки и тестирования независимых компонентов системы (слой данных, бизнес-логика, пользовательский интерфейс).
- Возможность на каждом этапе получать рабочий прототип и демонстрировать его руководителю практики для оперативного получения обратной связи.
- Снижение рисков благодаря раннему выявлению проблем и ошибок на каждой итерации.

Процесс разработки был разбит на следующие итерации:

1. Проектирование базы данных и создание скриптов.
2. Создание моделей данных, контекста БД и репозиториев.
3. Реализация слоя сервисов (бизнес-логика, расчет скидок).
4. Разработка главного окна и отображение списка партнеров.
5. Разработка окна добавления и редактирования партнера.
6. Разработка окна истории продаж.
7. Интеграция модулей, отладка и тестирование.

Данный подход позволил на каждом этапе получать рабочий прототип с новой функциональностью и оперативно вносить коррективы без необходимости переработки всего проекта.

2. Инструменты, использованные для создания продукта

Для разработки программного продукта был использован следующий набор инструментов:

Среда разработки (IDE): Microsoft Visual Studio 2022 Community Edition. Выбор данной среды обусловлен наличием всех необходимых инструментов для разработки на платформе .NET, включая поддержку языков C# и XAML, встроенный дизайнер интерфейсов WPF, мощный отладчик, средства рефакторинга и навигации по коду, а также интеграцию с Git.

Язык программирования: C# — современный объектно-ориентированный язык, являющийся основным для платформы .NET. Выбор языка обусловлен его строгой типизацией, богатой стандартной библиотекой и широкими возможностями для создания приложений с графическим интерфейсом.

Технология для создания интерфейса: WPF (Windows Presentation Foundation) с использованием паттерна MVVM (Model-View-ViewModel). Данная технология позволяет создавать насыщенные графические интерфейсы с использованием языка разметки XAML и разделять логику интерфейса (View) от логики представления данных (ViewModel) и бизнес-логики (Model).

Фреймворк для работы с базами данных: Entity Framework Core (ORM). Данный фреймворк позволяет взаимодействовать с базой данных через объекты C#, автоматически генерировать SQL-запросы и управлять подключением, что значительно упрощает разработку и уменьшает объем кода, необходимого для работы с данными.

Система управления базами данных (СУБД): PostgreSQL. Выбор обусловлен требованиями методических указаний, а также надежностью,

производительностью, бесплатным распространением и поддержкой стандартов SQL данной СУБД.

Система контроля версий: Git и платформа GitHub (описано в п. 3.2.1).

Пакетный менеджер: NuGet для управления зависимостями проекта, добавления сторонних библиотек и их обновления.

Фреймворк для тестирования: NUnit для создания и выполнения модульных тестов.

3. Описание среды программирования

Среда программирования, в которой осуществлялась разработка, представляет собой комплекс программных средств, обеспечивающих комфортную и эффективную работу над проектом. Основным компонентом среды является Microsoft Visual Studio 2022 Community Edition — интегрированная среда разработки (IDE), которая предоставляет следующие возможности:

Редактор кода с поддержкой подсветки синтаксиса для C# и XAML, автодополнением (IntelliSense), быстрой навигацией по коду, рефакторингом и форматированием. Редактор позволяет одновременно работать с несколькими файлами, открытыми в разных вкладках или разделенных окнах.

Дизайнер интерфейсов XAML предоставляет возможность визуального проектирования пользовательского интерфейса с возможностью перетаскивания элементов управления на форму и одновременного просмотра генерируемого кода разметки. Это ускоряет процесс создания окон и упрощает настройку расположения элементов.

Обозреватель решений (Solution Explorer) отображает структуру проекта, позволяет управлять файлами, добавлять новые элементы, устанавливать пакеты NuGet и настраивать ссылки между проектами.

Отладчик является мощным инструментом для поиска и устранения ошибок. Он позволяет устанавливать точки останова, выполнять пошаговое выполнение кода, отслеживать значения переменных, анализировать стек вызовов и управлять потоком выполнения программы.

Интеграция с Git обеспечивает выполнение основных операций с системой контроля версий непосредственно из среды разработки: просмотр изменений, создание коммитов, синхронизацию с удаленным репозиторием, просмотр истории.

Окно управления пакетами NuGet позволяет легко находить, устанавливать и обновлять сторонние библиотеки, необходимые для проекта.

4. Построение диаграммы классов

Архитектура разработанного приложения построена с использованием паттерна MVVM и принципов многослойной архитектуры. Диаграмма классов, созданная средствами Visual Studio, отражает основные сущности проекта и их взаимосвязи.

Проект разделен на несколько логических слоев:

Модели (Models). Данный слой содержит классы, представляющие сущности предметной области и соответствующие таблицам базы данных. Основные модели включают:

- Partner (Рисунок 1) — содержит данные о партнере: идентификатор, тип партнера, наименование компании, юридический адрес, ИНН, ФИО директора, телефон, email, рейтинг, путь к логотипу, даты создания и обновления записи, а также навигационные свойства для связанных сущностей.

```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

namespace SadiullinaLibrary.Models
{
    [Table("partners_sadiullina", Schema = "app")]
    Ссылок: 36
    public class Partner
    {
        [Key]
        [Column("id")]
        Ссылок: 16 | 7/7 passing
        public int Id { get; set; }

        [Column("type_id")]
        Ссылок: 14 | 7/7 passing
        public int TypeId { get; set; }

        [Required]
        [Column("company_name")]
        [MaxLength(255)]
        Ссылок: 19 | 8/8 passing
        public string CompanyName { get; set; } = string.Empty;

        [Column("legal_address")]
        Ссылок: 4
        public string? LegalAddress { get; set; }

        [Column("inn")]
        [MaxLength(12)]
        Ссылок: 5
        public string? Inn { get; set; }

        [Column("director_fullname")]
        [MaxLength(255)]
        Ссылок: 6 | 2/2 passing
        public string? DirectorFullname { get; set; }

        [Column("phone")]
        [MaxLength(20)]
        Ссылок: 6 | 2/2 passing
        public string? Phone { get; set; }

        [Column("email")]
        [MaxLength(100)]
        Ссылок: 6 | 2/2 passing
        public string? Email { get; set; }
```

Рисунок 1 — Класс Partner.

- PartnerType (Рисунок 2) — справочник типов партнеров с полями идентификатора, наименования и описания.

```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

namespace SadiullinaLibrary.Models
{
    [Table("partner_types_sadiullina", Schema = "app")]
    Ссылок: 14
    public class PartnerType
    {
        [Key]
        [Column("id")]
        Ссылок: 5 | 1/1 passing
        public int Id { get; set; }

        [Required]
        [Column("name")]
        [MaxLength(100)]
        Ссылок: 5 | 1/1 passing
        public string Name { get; set; } = string.Empty;

        [Column("description")]
        Ссылок: 0
        public string? Description { get; set; }

        Ссылок: 1
        public virtual ICollection<Partner> Partners { get; set; } = new List<Partner>();
    }
}
```

Рисунок 2 — Класс PartnerType.

- Product (Рисунок 3) — модель продукции с артикулом, наименованием, описанием и другими характеристиками.

```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

namespace SadiullinaLibrary.Models
{
    [Table("products_sadiullina", Schema = "app")]
    Ссылок: 7
    public class Product
    {
        [Key]
        [Column("id")]
        Ссылок: 0
        public int Id { get; set; }

        [Required]
        [Column("article")]
        [MaxLength(50)]
        Ссылок: 2
        public string Article { get; set; } = string.Empty;

        [Required]
        [Column("name")]
        [MaxLength(255)]
        Ссылок: 2
        public string Name { get; set; } = string.Empty;

        [Column("description")]
        Ссылок: 0
        public string? Description { get; set; }

        [Column("min_price")]
        Ссылок: 0
        public decimal? MinPrice { get; set; }

        [Column("unit")]
        [MaxLength(20)]
        Ссылок: 0
        public string? Unit { get; set; }

        [Column("created_at")]
        Ссылок: 0
        public DateTime CreatedAt { get; set; }

        Ссылок: 1
        public virtual ICollection<SalesHistory> SalesHistories { get; set; } = new List<SalesHistory>();
```

Рисунок 3 — Класс Product.

- SalesHistory (Рисунок 4) — модель истории продаж, содержащая ссылки на партнера и продукт, количество, дату продажи и общую сумму.

```
namespace SadiullinaLibrary.Models
{
    [Table("sales_history_sadiullina", Schema = "app")]
    Ссылок: 10
    public class SalesHistory
    {
        [Key]
        [Column("id")]
        Ссылок: 1
        public int Id { get; set; }

        [Column("partner_id")]
        Ссылок: 3
        public int PartnerId { get; set; }

        [Column("product_id")]
        Ссылок: 1
        public int ProductId { get; set; }

        [Column("quantity")]
        Ссылок: 1
        public int Quantity { get; set; }

        [Column("sale_date")]
        Ссылок: 3
        public DateTime SaleDate { get; set; }

        [Column("total_amount")]
        Ссылок: 2
        public decimal TotalAmount { get; set; }

        [Column("created_at")]
        Ссылок: 0
        public DateTime CreatedAt { get; set; }

        [ForeignKey("PartnerId")]
        Ссылок: 1
        public virtual Partner? Partner { get; set; }

        [ForeignKey("ProductId")]
        Ссылок: 4
        public virtual Product? Product { get; set; }
    }
```

Рисунок 4 — Класс SalesHistory.

- SalesPoint (Рисунок 5) — модель места продаж партнера с адресом и типом точки.

```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

namespace SadiullinaLibrary.Models
{
    [Table("sales_points_sadiullina", Schema = "app")]
    Ссылок: 3
    public class SalesPoint
    {
        [Key]
        [Column("id")]
        Ссылок: 0
        public int Id { get; set; }

        [Column("partner_id")]
        Ссылок: 0
        public int PartnerId { get; set; }

        [Required]
        [Column("address")]
        Ссылок: 0
        public string Address { get; set; } = string.Empty;

        [Column("point_type")]
        [MaxLength(50)]
        Ссылок: 0
        public string? PointType { get; set; }

        [ForeignKey("PartnerId")]
        Ссылок: 0
        public virtual Partner? Partner { get; set; }
    }
}
```

Рисунок 5 — Класс SalesPoint.

Data Layer. Содержит классы для взаимодействия с базой данных:

- AppDbContext (Рисунок 6) — контекст базы данных Entity Framework Core, который управляет подключением к БД, содержит наборы данных (DbSet<T>) и настройки моделирования (переопределение метода OnModelCreating).

```
using SadiullinaLibrary.Models;

namespace SadiullinaLibrary.Data
{
    public class AppDbContext : DbContext
    {
        public AppDbContext(DbContextOptions<AppDbContext> options)
            : base(options)
        {
        }

        public DbSet<PartnerType> PartnerTypes { get; set; }

        public DbSet<Partner> Partners { get; set; }

        public DbSet<Product> Products { get; set; }

        public DbSet<SalesHistory> SalesHistories { get; set; }

        public DbSet<SalesPoint> SalesPoints { get; set; }

        protected override void OnModelCreating(ModelBuilder modelBuilder)
        {
            base.OnModelCreating(modelBuilder);

            modelBuilder.HasDefaultSchema("app");
```

Рисунок 6 — Класс AppDbContext.

- Репозитории (PartnerRepository, PartnerTypeRepository, ProductRepository) с соответствующими интерфейсами (IPartnerRepository, IPartnerTypeRepository), реализующие паттерн "Репозиторий" для абстрагирования логики доступа к данным (Рисунок 7).

```
using SadiullinaLibrary.Models;

namespace SadiullinaLibrary.Data
{
    Ссылок: 6
    public interface IPartnerRepository
    {
        Ссылок: 4 | 2/2 passing
        Task<List<Partner>> GetAllAsync();
        Ссылок: 10 | 6/6 passing
        Task<Partner?> GetByIdAsync(int id);
        Ссылок: 10 | 6/6 passing
        Task<Partner> AddAsync(Partner partner);
        Ссылок: 3 | 1/1 passing
        Task UpdateAsync(Partner partner);
        Ссылок: 3 | 1/1 passing
        Task DeleteAsync(int id);
        Ссылок: 2
        Task<List<SalesHistory>> GetPartnerSalesHistoryAsync(int partnerId);
        Ссылок: 4 | 1/1 passing
        Task<decimal> GetTotalSalesAmountByPartnerAsync(int partnerId);
    }

    Ссылок: 4
    public class PartnerRepository : IPartnerRepository
    {
        private readonly AppDbContext _context;

        Ссылок: 1
        public PartnerRepository(AppDbContext context)
        {
            _context = context;
        }

        Ссылок: 4 | 2/2 passing
        public async Task<List<Partner>> GetAllAsync()
        {
            return await _context.Partners
                .Include(p => p.PartnerType)
                .OrderBy(p => p.CompanyName)
                .ToListAsync();
        }
```

Рисунок 7 — Класс PartnerRepository.

Сервисы (Services). Содержат бизнес-логику приложения:

- PartnerService (Рисунок 8) — координирует работу с данными партнеров, используя репозитории для получения данных и преобразуя их в DTO, а также вызывает алгоритм расчета скидок.

```
using SadiullinaLibrary.Models;

namespace SadiullinaLibrary.Services
{
    Ссылок: 8
    public interface IPartnerService
    {
        Ссылок: 3 | 1/1 passing
        Task<List<PartnerDto>> GetAllPartnersAsync();
        Ссылок: 4 | 2/2 passing
        Task<PartnerCreateUpdateDto?> GetPartnerForEditAsync(int id);
        Ссылок: 1
        Task<Partner?> GetPartnerByIdAsync(int id);
        Ссылок: 3 | 1/1 passing
        Task<Partner> CreatePartnerAsync(PartnerCreateUpdateDto dto);
        Ссылок: 2
        Task UpdatePartnerAsync(PartnerCreateUpdateDto dto);
        Ссылок: 2
        Task DeletePartnerAsync(int id);
        Ссылок: 3
        Task<List<SalesHistoryDto>> GetPartnerSalesHistoryAsync(int partnerId);
    }

    Ссылок: 4
    public class PartnerService : IPartnerService
    {
        private readonly IPartnerRepository _partnerRepository;
        private readonly IProductRepository _productRepository;
        private readonly IPartnerTypeRepository _partnerTypeRepository;
        private readonly IDiscountCalculator _discountCalculator;

        Ссылок: 1
        public PartnerService(
            IPartnerRepository partnerRepository,
            IProductRepository productRepository,
            IPartnerTypeRepository partnerTypeRepository,
            IDiscountCalculator discountCalculator)
        {
```

Рисунок 8 — Класс PartnerService.

- DiscountCalculator (Рисунок 9)— инкапсулирует алгоритм расчета индивидуальной скидки на основе общей суммы продаж.

```
namespace SadiullinaLibrary.Services
{
    Ссылок: 6
    public interface IDiscountCalculator
    {
        Ссылок: 10 | 12/12 passing
        int CalculateDiscount(decimal totalSalesAmount);
    }

    Ссылок: 4
    public class DiscountCalculator : IDiscountCalculator
    {
        Ссылок: 10 | 12/12 passing
        public int CalculateDiscount(decimal totalSalesAmount)
        {
            if (totalSalesAmount < 10000)
                return 0;
            else if (totalSalesAmount < 50000)
                return 5;
            else if (totalSalesAmount < 300000)
                return 10;
            else
                return 15;
        }
    }
}
```

Рисунок 9 — Класс DiscountCalculator.

DTO (Data Transfer Objects). Классы для передачи данных между слоями приложения без раскрытия внутренней структуры моделей БД:

- PartnerDto (Рисунок 10) — содержит данные для отображения в списке партнеров, включая рассчитанную скидку.

```
namespace SadiullinaLibrary.DTO
{
    Ссылок: 12
    public class PartnerDto
    {
        Ссылок: 9 | 1/1 passing
        public int Id { get; set; }
        Ссылок: 1
        public int TypeId { get; set; }
        Ссылок: 1
        public string TypeName { get; set; } = string.Empty;
        Ссылок: 4
        public string CompanyName { get; set; } = string.Empty;
        Ссылок: 1
        public string? LegalAddress { get; set; }
        Ссылок: 1
        public string? Inn { get; set; }
        Ссылок: 2
        public string? DirectorFullname { get; set; }
        Ссылок: 2
        public string? Phone { get; set; }
        Ссылок: 1
        public string? Email { get; set; }
        Ссылок: 1
        public int Rating { get; set; }
        Ссылок: 1
        public decimal TotalSalesAmount { get; set; }
        Ссылок: 4 | 1/1 passing
        public int DiscountPercentage { get; set; }
        Ссылок: 0
        public string DiscountDisplay => $"Скидка: {DiscountPercentage}%";
    }
}
```

Рисунок 10 — Класс PartnerDto.

- SalesHistoryDto (Рисунок 11)— содержит данные для отображения истории продаж.

```
namespace SadiullinaLibrary.DTO
{
    Ссылок: 9
    public class SalesHistoryDto
    {
        Ссылок: 1
        public int Id { get; set; }
        Ссылок: 1
        public string ProductName { get; set; } = string.Empty;
        Ссылок: 1
        public string ProductArticle { get; set; } = string.Empty;
        Ссылок: 1
        public int Quantity { get; set; }
        Ссылок: 2
        public DateTime SaleDate { get; set; }
        Ссылок: 3
        public decimal TotalAmount { get; set; }
        Ссылок: 0
        public string FormattedSaleDate => SaleDate.ToString("dd.MM.yyyy");
        Ссылок: 0
        public string FormattedTotalAmount => TotalAmount.ToString("N2") + " ₽";
    }
}
```

Рисунок 11 — Класс SalesHistoryDto.

- PartnerCreateUpdateDto (Рисунок 12)— используется для передачи данных при создании и редактировании партнера.

```
using System.ComponentModel.DataAnnotations;

namespace SadiullinaLibrary.DTO
{
    Ссылок: 12
    public class PartnerCreateUpdateDto
    {
        Ссылок: 5 ● 1/1 passing
        public int? Id { get; set; }

        [Required(ErrorMessage = "Тип партнера обязателен")]
        Ссылок: 7 ● 1/1 passing
        public int TypeId { get; set; }

        [Required(ErrorMessage = "Наименование обязательно")]
        [StringLength(255, MinimumLength = 2, ErrorMessage = "Наименование должно быть от 2 до 255 символов")]
        Ссылок: 6 ● 2/2 passing
        public string CompanyName { get; set; } = string.Empty;

        Ссылок: 3
        public string? LegalAddress { get; set; }

        [RegularExpression(@"^\d{10,12}$", ErrorMessage = "ИНН должен содержать 10-12 цифр")]
        Ссылок: 3
        public string? Inn { get; set; }

        Ссылок: 4 ● 1/1 passing
        public string? DirectorFullname { get; set; }

        [RegularExpression(@"^\+?[\d\s\-\(\)]{10,20}$", ErrorMessage = "Введите корректный номер телефона")]
        Ссылок: 3
        public string? Phone { get; set; }

        [EmailAddress(ErrorMessage = "Введите корректный email")]
        Ссылок: 3
        public string? Email { get; set; }

        [Range(0, 1000, ErrorMessage = "Рейтинг должен быть от 0 до 1000")]
        Ссылок: 7 ● 2/2 passing
        public int Rating { get; set; }
    }
```

Рисунок 12 — Класс PartnerCreateUpdateDto.

ViewModels. Содержат логику представления для каждого окна, реализуют интерфейс INotifyPropertyChanged для уведомления представления об изменениях данных и предоставляют команды для обработки действий пользователя:

- MainViewModel (Рисунок 13)— отвечает за главное окно, содержит коллекции партнеров и истории продаж, команды для добавления, редактирования, удаления и просмотра истории.

```
using System.Windows;
using System.Windows.Input;

namespace Sadiullina.ViewModels
{
    Ссылок: 5
    public class MainViewModel : ViewModelBase
    {
        private readonly IPartnerService _partnerService;

        private ObservableCollection<PartnerDto> _partners;
        private ObservableCollection<SalesHistoryDto> _selectedPartnerOrders;
        private PartnerDto? _selectedPartner;
        private bool _isLoading;
        private string _searchText = string.Empty;

        Ссылок: 0
        public MainViewModel(IPartnerService partnerService)
        {
            _partnerService = partnerService;
            _partners = new ObservableCollection<PartnerDto>();
            _selectedPartnerOrders = new ObservableCollection<SalesHistoryDto>();

            InitializeCommands();
            LoadPartnersAsync();
```

Рисунок 13 — Класс MainViewModel.

- PartnerEditViewModel (Рисунок 14)— отвечает за окно добавления и редактирования партнера, содержит текущий объект партнера, список типов партнеров, команды сохранения и отмены.

```
using System.Windows.Input;
using System.Threading.Tasks;

namespace Sadiullina.ViewModels
{
    Ссылок: 6
    public class PartnerEditViewModel : ViewModelBase
    {
        private readonly IPartnerService _partnerService;
        private readonly IPartnerTypeRepository _partnerTypeRepository;

        private PartnerCreateUpdateDto _currentPartner;
        private ObservableCollection<PartnerType> _partnerTypes;
        private PartnerType? _selectedType;
        private bool _isEditMode;
        private bool _isLoading;
        private bool _isInitialized = false;

        Ссылок: 0
        public PartnerEditViewModel(
            IPartnerService partnerService,
            IPartnerTypeRepository partnerTypeRepository)
        {
            _partnerService = partnerService;
            _partnerTypeRepository = partnerTypeRepository;
            _currentPartner = new PartnerCreateUpdateDto();
            _partnerTypes = new ObservableCollection<PartnerType>();

            InitializeCommands();
```

Рисунок 14 — Класс PartnerEditViewModel.

- SalesHistoryViewModel (Рисунок 15)— отвечает за окно истории продаж, содержит коллекцию продаж и логику расчета скидки.

```
namespace Sadiullina.ViewModels
{
    Ссылок: 2
    public class SalesHistoryViewModel : ViewModelBase
    {
        private readonly IPartnerService _partnerService;
        private readonly int _partnerId;

        private ObservableCollection<SalesHistoryDto> _salesHistory;
        private bool _isLoading;
        private decimal _totalSalesAmount;
        private int _discountPercentage;

        Ссылок: 1
        public SalesHistoryViewModel(IPartnerService partnerService, int partnerId)
        {
            _partnerService = partnerService;
            _partnerId = partnerId;
            _salesHistory = new ObservableCollection<SalesHistoryDto>();

            LoadSalesHistoryAsync();
        }

        Ссылок: 2
        public ObservableCollection<SalesHistoryDto> SalesHistory
        {
            get => _salesHistory;
            set => SetProperty(ref _salesHistory, value);
        }

        Ссылок: 2
        public bool IsLoading
        {
            get => _isLoading;
            set => SetProperty(ref _isLoading, value);
        }
```

Рисунок 15 — Класс SalesHistoryViewModel.

Views. XAML-разметка окон, отвечающая только за отображение данных и привязку к командам и свойствам из ViewModel:

- MainWindow.xaml
- PartnerEditWindow.xaml
- SalesHistoryWindow.xaml

Converters. Вспомогательные классы для преобразования данных при привязке в XAML, например, NullableIntToStringConverter для преобразования nullable-целых чисел в строки и обратно.

Диаграмма классов графически представлена на Рисунке 15.

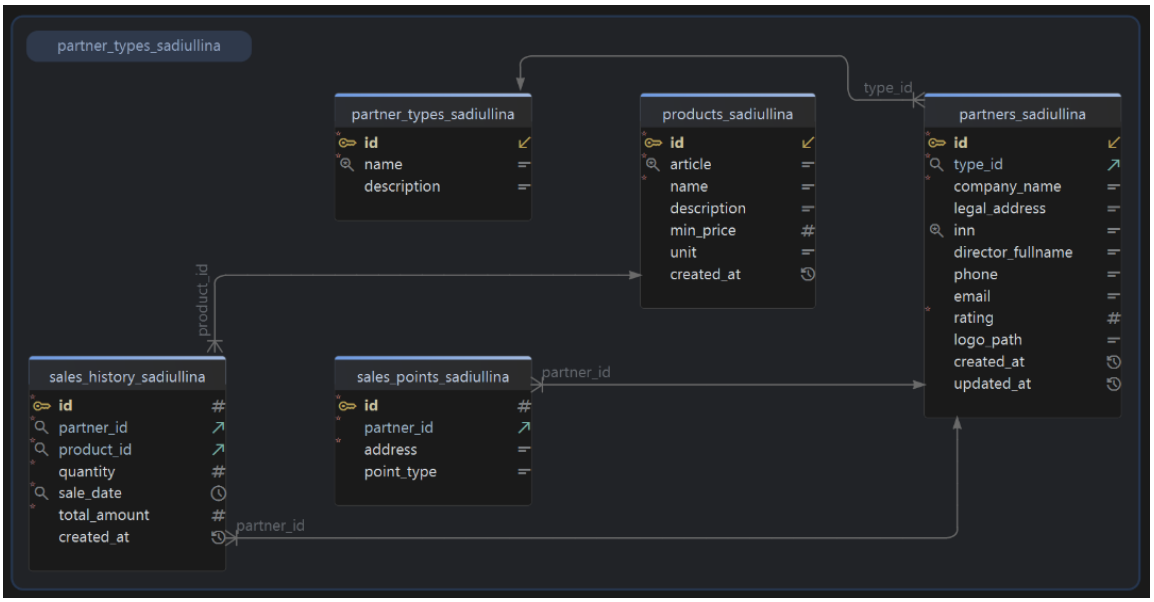

Рисунок 15 — Диаграмма классов программного модуля.

5. Разработка программного модуля согласно требованиям

Подсистема должна обеспечивать просмотр списка партнеров, добавление и редактирование данных о партнере, просмотр истории реализации продукции партнером, а также расчет индивидуальной скидки на основе объема продаж.

В ходе выполнения производственного задания были реализованы следующие функциональные требования:

Просмотр списка партнеров. На главной форме приложения отображается список всех партнеров, информация о которых хранится в базе данных. Каждый партнер представлен в виде карточки, содержащей наименование компании, ФИО директора, телефон, рейтинг, а также рассчитанную на основе объема продаж индивидуальную скидку. При выборе партнера в правой панели отображается история реализации его продукции.

Добавление и редактирование данных о партнере. Реализована форма для добавления и редактирования партнера, которая открывается из главного меню или контекстного меню. Форма содержит все необходимые поля:

наименование, тип партнера (выпадающий список), рейтинг, адрес, ФИО директора, телефон и email компании. Рейтинг партнера является целым неотрицательным числом. При открытии формы для редактирования все поля выбранного объекта заполняются актуальными данными из базы данных. После сохранения изменений данные в окне списка партнеров обновляются автоматически.

Просмотр истории реализации продукции. При выборе партнера в главном окне в правой панели отображается история реализации продукции с указанием наименования продукции, количества и даты продажи. Для просмотра детальной информации реализовано отдельное окно истории продаж, которое также отображает общую сумму продаж и рассчитанную скидку.

Расчет индивидуальной скидки. Реализован алгоритм расчета скидки для партнера на основании суммарного объема его продаж согласно заданной шкале:

- до 10 000 руб. – 0%;
- от 10 000 до 50 000 руб. – 5%;
- от 50 000 до 300 000 руб. – 10%;
- более 300 000 руб. – 15%.

6. Этапы разработки программного обеспечения

Процесс разработки программного обеспечения включал следующие этапы:

Выбор шаблона и настройка проекта. На начальном этапе был создан проект WPF с именем Sadiullina (содержит фамилию студента согласно требованиям методических указаний). Отдельно была создана библиотека классов .NET с именем SadiullinaLibrary для размещения моделей, слоя данных и сервисов, что обеспечило модульность архитектуры и разделение ответственности между компонентами. Были настроены необходимые пакеты NuGet (Рисунок 16):

- Microsoft.EntityFrameworkCore,

- Npgsql.EntityFrameworkCore.PostgreSQL,
- Microsoft.Extensions.Configuration.Json,
- Microsoft.Extensions.DependencyInjection.

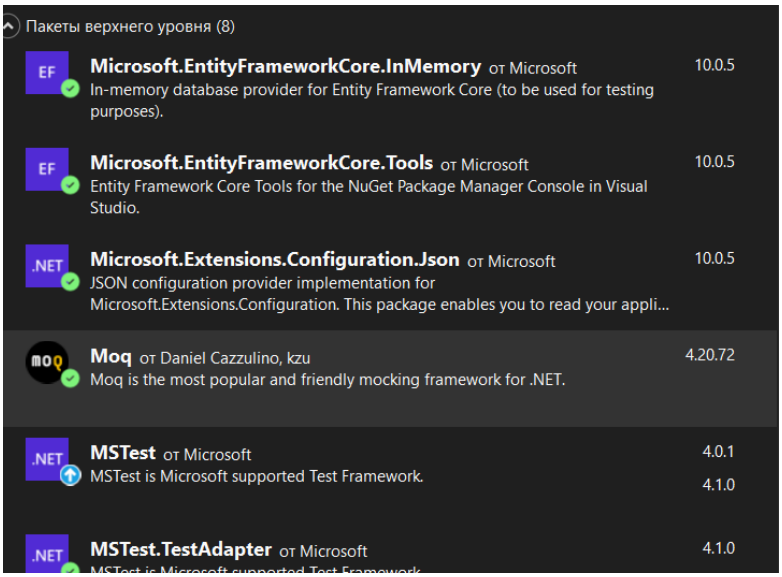

Рисунок 16 — Пакеты NuGet.

Составление схемы информационной базы. Была спроектирована база данных, состоящая из пяти таблиц: partner_types_sadiullina, partners_sadiullina, products_sadiullina, sales_history_sadiullina, sales_points_sadiullina. Между таблицами установлены связи "один-ко-многим". Создан скрипт базы данных (Sadiullina_DB_Script.sql), учитывающий все требования к именованию (схема app, фамилия студента в названиях таблиц) и ссылочной целостности. Схема базы данных задокументирована в виде ER-диаграммы, созданной средствами pgAdmin.

Документирование системы. В процессе разработки велась техническая документация, включающая описание архитектуры приложения, структуры базы данных, назначения классов и методов. В коде проекта расставлены комментарии, поясняющие неочевидные фрагменты и сложные алгоритмические решения.

Тестирование (Рисунок 17). Проводилось модульное тестирование ключевых компонентов (алгоритм расчета скидок) и интеграционное тестирование взаимодействия всех модулей системы.

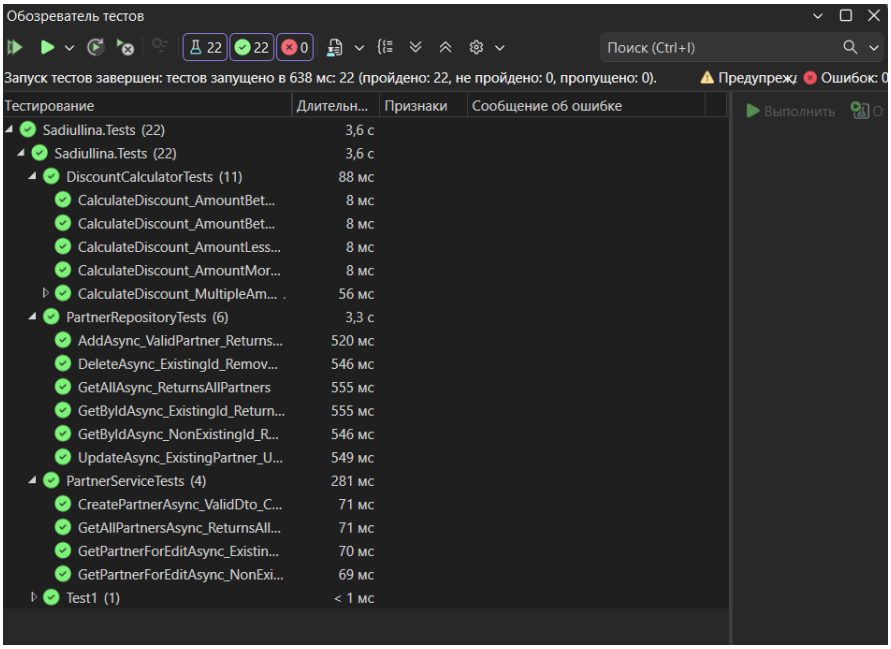

Рисунок 17 – MS Тестирование.

Сопровождение системы. На этапе сопровождения осуществлялся анализ обратной связи от руководителя практики, исправление выявленных недочетов и оптимизация работы приложения.

7. Разработка форм для ввода, просмотра и редактирования данных

В ходе выполнения задания были разработаны следующие формы пользовательского интерфейса:

Главное окно (MainWindow.xaml). Реализует просмотр списка партнеров в виде карточек в соответствии с эскизом, представленным в задании (Рисунок 18). Список привязан к коллекции Partners из MainViewModel. Для каждого партнера отображается наименование, ФИО директора, телефон, рейтинг и рассчитанная скидка. При выборе партнера цвет его карточки изменяется (стиль SelectedPartnerCardStyle), а в правой части окна отображается история его продаж, привязанная к коллекции SelectedPartnerOrders. Реализовано главное меню с пунктами для работы с партнерами (Файл, Партнеры, Справка), а также контекстное меню, вызываемое правой кнопкой мыши на карточке партнера.

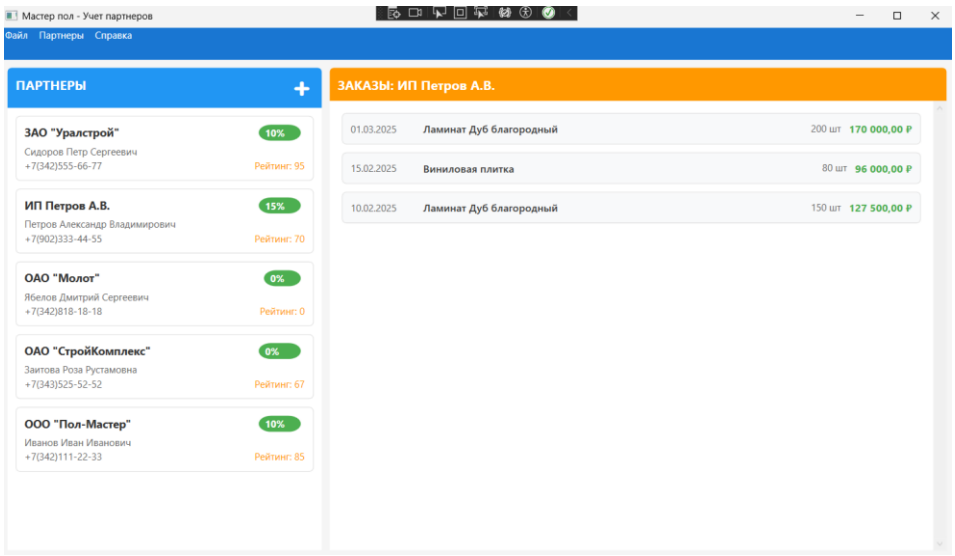

Рисунок 18 – MainWindow.

Окно добавления и редактирования партнера (PartnerEditWindow.xaml). Содержит все необходимые поля согласно заданию:

- Выпадающий список для выбора типа партнера, источником данных для которого служит коллекция PartnerTypes из PartnerEditViewModel.
- Текстовые поля для ввода наименования, юридического адреса, ИНН, ФИО директора, телефона, email.
- Поле для ввода рейтинга, ограниченное целыми неотрицательными числами (валидация с помощью атрибута Range в DTO).

При открытии в режиме редактирования все поля заполняются данными выбранного партнера, полученными из базы данных через сервис. Реализована проверка обязательных полей (помечены символом "*"). Кнопка "Сохранить" активируется только при выполнении условий валидации. После успешного сохранения окно закрывается, и данные в главном окне обновляются (Рисунок 19).

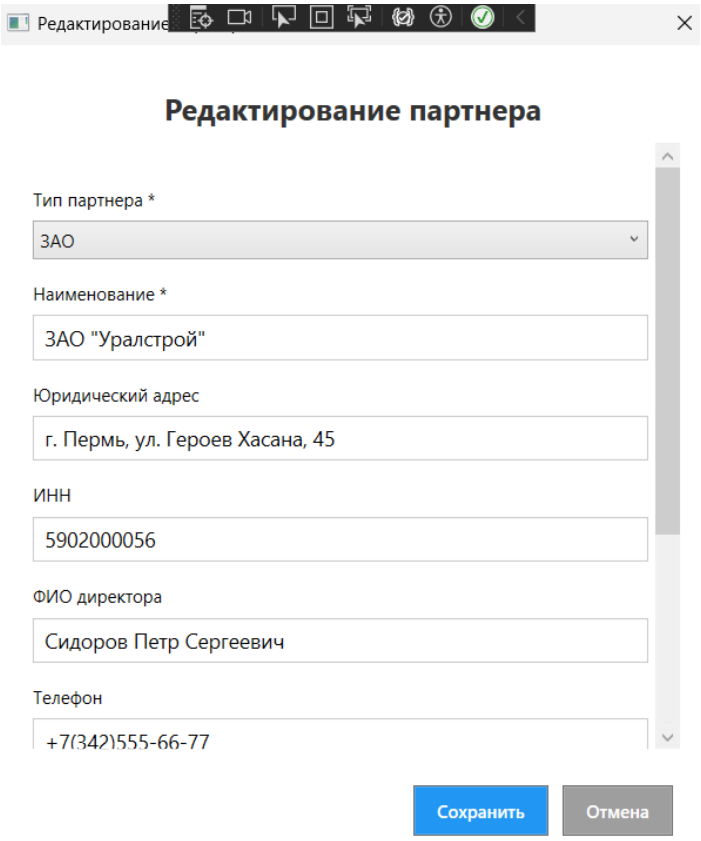

Рисунок 19 – PartnerEditWindow.

Окно истории продаж (SalesHistoryWindow.xaml). Отображает детальную информацию о продажах выбранного партнера в виде таблицы (DataGrid) с колонками: дата продажи, наименование продукции, артикул, количество, сумма. Дополнительно выводится информационная панель, показывающая общую сумму всех продаж партнера и рассчитанную на ее основе индивидуальную скидку. Данные для отображения привязаны к коллекции SalesHistory из SalesHistoryViewModel (Рисунок 20).

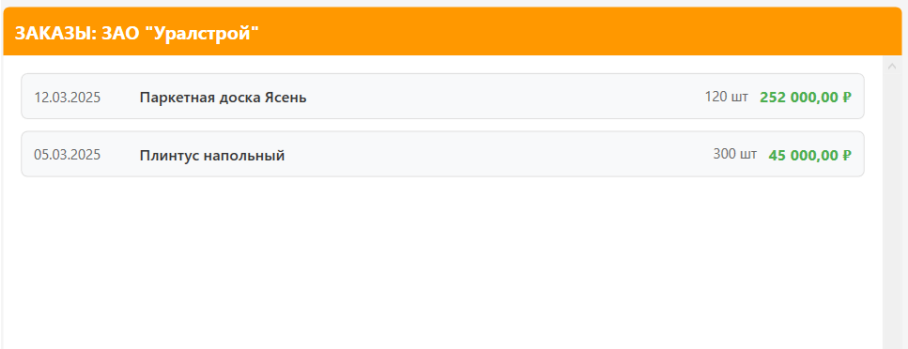

Рисунок 20.1. – SalesHistoryWindow.

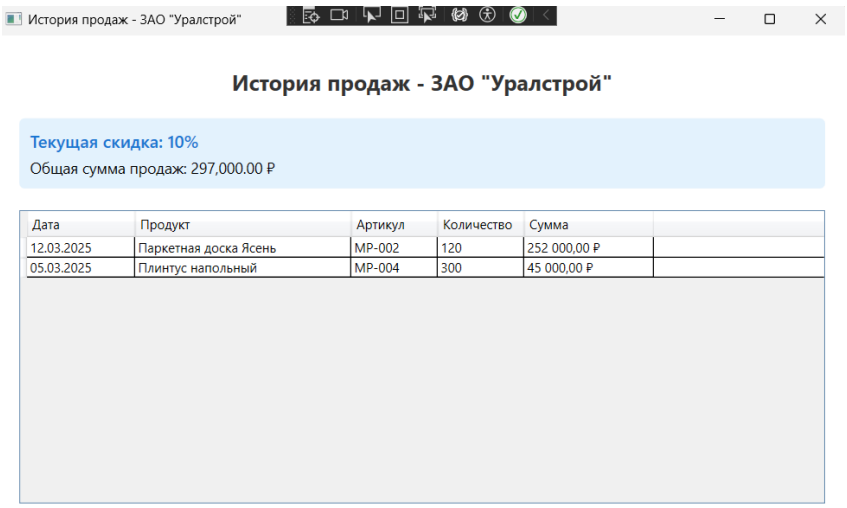

Рисунок 20.2. – SalesHistoryWindow.

8. Этапы разработки интерфейса

Разработка пользовательского интерфейса включала следующие этапы:

Проектирование. На основе эскизов, предоставленных в задании, была спроектирована структура окон и навигация между ними. Для главного окна была выбрана двухпанельная компоновка, позволяющая одновременно отображать список партнеров слева и детальную информацию о выбранном партнере (историю продаж) справа. Определен состав элементов управления для каждого окна и их расположение.

Прототипирование. В среде Visual Studio с использованием языка разметки XAML был создан грубый прототип интерфейса с применением стандартных элементов управления без детальной стилизации. На этом этапе проверялась удобство расположения элементов, корректность работы привязок данных и навигации между окнами. Прототип позволил выявить и исправить недостатки компоновки до начала детальной проработки дизайна.

Стилизация. Для придания приложению современного и профессионального вида, а также для обеспечения соответствия единому стилю системы, были созданы и применены стили в файле App.xaml. Стилизованы основные элементы управления:

- PrimaryButtonStyle для кнопок с синим фоном, белым текстом и эффектом при наведении.
- TextBoxStyle для полей ввода с отступами и рамкой.

- HeaderStyle для заголовков окон.
- PartnerCardStyle и SelectedPartnerCardStyle для карточек партнеров с изменением внешнего вида при наведении мыши и выборе.

Цветовая гамма (оттенки синего и оранжевого) выбрана для создания приятного визуального опыта и соответствия корпоративному стилю условной компании.

3.2.3. Использование методов для получения кода с заданной функциональностью и степенью качества

Для обеспечения требуемой функциональности и достижения высокого качества кода применялись следующие методы и подходы:

Применение паттернов проектирования. Использование паттерна MVVM позволило достичь четкого разделения ответственности между компонентами приложения. View отвечает только за отображение, ViewModel — за логику представления и состояние, Model — за данные и бизнес-логику. Такое разделение обеспечивает слабую связанность компонентов, упрощает тестирование и модификацию кода. Применение паттерна Репозиторий (Repository) абстрагировало слой доступа к данным, позволяя изолировать логику работы с базой данных от остальной части приложения. Паттерн Внедрение зависимостей (Dependency Injection) использован для обеспечения слабой связанности и повышения тестируемости кода.

Объектно-ориентированное программирование (ООП). Активно использовались основные принципы ООП:

- Инкапсуляция реализована через сокрытие внутреннего состояния классов и предоставление доступа к нему только через публичные методы и свойства.
- Наследование применено при создании базового класса ViewModelBase, от которого наследуются все конкретные ViewModel, что позволило избежать дублирования кода для реализации INotifyPropertyChanged.

- Полиморфизм реализован через использование интерфейсов (IPartnerService, IPartnerRepository), что позволяет подменять реализации, например, при модульном тестировании.

Принципы SOLID. Архитектура приложения разработана с учетом принципов SOLID:

- Принцип единственной ответственности (Single Responsibility Principle): каждый класс имеет только одну причину для изменения. Например, DiscountCalculator отвечает только за расчет скидки, PartnerRepository — только за доступ к данным о партнерах.
- Принцип открытости/закрытости (Open/Closed Principle): классы открыты для расширения, но закрыты для модификации. Новые типы партнеров могут быть добавлены без изменения существующего кода.
- Принцип подстановки Лисков (Liskov Substitution Principle): производные классы могут быть использованы вместо базовых без нарушения логики.
- Принцип разделения интерфейса (Interface Segregation Principle): интерфейсы являются узкоспециализированными (например, IPartnerRepository, IPartnerTypeRepository).
- Принцип инверсии зависимостей (Dependency Inversion Principle): зависимости направлены на абстракции (интерфейсы), а не на конкретные реализации, что реализовано через внедрение зависимостей в конструкторы классов.

Использование Data Transfer Objects (DTO). Применение DTO позволило передавать между слоями приложения только необходимые данные, избегая передачи всей внутренней структуры моделей БД. Это также позволило включать в объекты для передачи вычисляемые поля, отсутствующие в моделях (например, DiscountPercentage в PartnerDto).

Именование идентификаторов в стиле CamelCase. В коде на C# строго соблюдалось соглашение об именовании: имена локальных переменных и параметров методов записывались в camelCase (начинаются со строчной

буквы), имена классов, методов, публичных свойств — в PascalCase (начинаются с заглавной буквы). Все идентификаторы именовались осмысленно, отражая назначение элемента.

Применение асинхронного программирования. Для операций, связанных с вводом-выводом (работа с базой данных), использовались асинхронные методы (async/await), что позволило избежать блокировки потока пользовательского интерфейса и повысить отзывчивость приложения.

Обработка исключений. В коде реализована обработка исключительных ситуаций с использованием блоков try-catch-finally. Пользователю выводятся информативные сообщения об ошибках с пояснением причин и возможных действий для их устранения.

3.2.4. Инспектирование компонент программного обеспечения на предмет соответствия стандартам кодирования

Инспектирование кода представляет собой процесс проверки исходного кода на предмет соответствия установленным стандартам и выявления потенциальных ошибок и недостатков.

1. Виды инспекции

В ходе разработки применялись следующие виды инспекции:

Формальная инспекция (самопроверка). Данный вид инспекции проводился разработчиком самостоятельно после реализации каждого функционального блока. Код просматривался на предмет соответствия стандартам оформления, наличия логических ошибок, избыточности, "запахов кода" (code smells) и потенциальных проблем производительности.

Технический обзор (peer-review). Код был предоставлен для анализа руководителю практики, который оценил его структуру, соответствие требованиям задания и соблюдение стандартов кодирования. Полученные замечания учитывались при доработке проекта.

2. Актуальные стандарты кодирования

При разработке на языке C# соблюдались общепринятые стандарты и рекомендации, изложенные в документации Microsoft и отраслевых руководствах по написанию качественного кода:

Соглашение об именовании. Соблюдались следующие правила именования:

- Имена классов, методов, свойств, событий, пространств имен записываются в PascalCase: PartnerService, LoadPartnersAsync, CompanyName.
- Имена локальных переменных и параметров методов записываются в camelCase: selectedPartner, totalSalesAmount.
- Имена закрытых полей начинаются с символа подчеркивания и записываются в camelCase: _partnerService, _selectedPartner.
- Имена интерфейсов начинаются с заглавной буквы I: IPartnerService, IPartnerRepository.

Все идентификаторы именуются осмысленно, полно и однозначно отражая назначение элемента. Использование сокращений допускается только для общепринятых аббревиатур.

3. Форматирование кода.

- Использовались стандартные отступы в 4 пробела.
- Фигурные скобки размещались на новой строке (стиль "Allman").
- Между логическими блоками кода оставлялись пустые строки для улучшения читаемости.
- Операторы отделялись пробелами.

Комментирование. Комментарии добавлялись только в местах, требующих дополнительного пояснения, например, для сложных алгоритмических решений, особенностей реализации, пояснения неочевидных фрагментов кода. Комментарии должны отвечать на вопрос "почему сделано так", а не "что делает код", так как последнее должно быть понятно из самого кода. Пример комментария: // Подписка на изменение выбранного партнера для загрузки истории продаж.

Структурирование кода. Код внутри классов организован логически: поля, конструктор, свойства, публичные методы, приватные методы. Это облегчает навигацию по файлам.

4. Программы для инспекции

Основным инструментом для инспекции кода служила сама среда разработки Microsoft Visual Studio 2022, которая обладает встроенными возможностями для анализа кода:

Анализатор кода (Code Analysis). Visual Studio выполняет непрерывный анализ кода по мере его написания, подсвечивая потенциальные проблемы, несоответствия стандартам и предлагая рекомендации по улучшению. Анализатор выявляет такие проблемы, как неиспользуемые переменные, возможные исключения, нарушения в именовании и другие.

Рефакторинг. Встроенные средства рефакторинга позволяют автоматически улучшать структуру кода без изменения его поведения: переименовывать идентификаторы, извлекать методы, изменять сигнатуры и т.д.

Подсветка синтаксиса и ошибок. Среда мгновенно сообщает о синтаксических ошибках, что позволяет исправлять их непосредственно в процессе написания кода.

Дополнительно для контроля качества кода использовались рекомендации, изложенные в книгах по написанию качественного кода на C# и стандартах кодирования Microsoft.

3.2.5. Выполнение отладки программного модуля с использованием специализированных программных средств

Отладка представляет собой процесс поиска, локализации и устранения ошибок в программном обеспечении. В ходе разработки приложения использовались различные методы и средства отладки, доступные в среде Visual Studio 2022.

1. Процесс проведения отладки

Процесс отладки включал следующие этапы и методы:

Установка точек останова (Breakpoints). На проблемных участках кода, где предполагалось наличие ошибок или требовалось проанализировать состояние программы, устанавливались точки останова. Для этого использовалось нажатие левой кнопкой мыши на левое поле редактора кода рядом со строкой. Точки останова устанавливались, например, в методе CalculateDiscount класса DiscountCalculator для проверки корректности расчета при различных входных значениях, в методе SaveAsync класса PartnerEditViewModel для проверки процесса сохранения данных, а также в методах загрузки данных из репозиториев.

Пошаговое выполнение (Stepping). После достижения точки останова применялись различные режимы пошагового выполнения:

- Step Over (F10) — выполнение текущей строки кода без входа в вызываемые методы. Использовался для быстрого прохождения по основному потоку выполнения, когда не требовалось детальное изучение вызываемых методов.
- Step Into (F11) — выполнение с входом в вызываемый метод. Применялся для детального анализа логики работы методов, особенно при возникновении неожиданного поведения.
- Step Out (Shift+F11) — выполнение до выхода из текущего метода. Использовался, когда детали реализации метода были уже понятны и требовалось вернуться к вызывающему коду.

Использование окон отладки. В процессе отладки активно применялись специализированные окна:

- Окно Locals автоматически отображало значения всех переменных в текущем контексте выполнения. Это позволяло быстро оценить состояние объектов и локальных переменных.
- Окно Watch использовалось для отслеживания значений конкретных переменных или выражений, которые требовали постоянного контроля.

- Окно Call Stack отображало последовательность вызовов методов, приведших к текущей точке выполнения. Это было особенно полезно при возникновении исключений для понимания контекста ошибки.
- Окно Immediate Window позволяло выполнять произвольные выражения и изменять значения переменных во время отладки для проверки различных сценариев без перезапуска приложения.

Анализ исключений (Exceptions). В Visual Studio была настроена остановка выполнения программы при возникновении исключений (даже обработанных). Это позволило выявить проблемы, которые иначе могли остаться незамеченными. Например, таким образом была обнаружена ошибка, связанная с нарушением уникальности индекса для поля Inn при попытке добавления партнера с уже существующим ИНН.

Проверка значений в DataTips. При наведении курсора мыши на переменную в режиме отладки появлялся DataTip, отображающий текущее значение переменной. Это быстрый способ проверки значений без открытия дополнительных окон.

В ходе отладки были выявлены и устранены следующие ошибки:

- Неправильный расчет скидки при граничных значениях (например, при сумме продаж ровно 50 000 рублей). Ошибка была исправлена корректировкой условий в методе CalculateDiscount.
- Некорректная загрузка связанных данных (тип партнера не отображался в списке). Проблема была решена добавлением Include(p => p.PartnerType) в запрос LINQ.
- Ошибка валидации на стороне клиента: поле рейтинга допускало ввод отрицательных значений. Исправлено добавлением атрибута Range(0, 1000) в DTO.
- Нарушение уникальности индекса при добавлении партнера с дублирующимся ИНН. Добавлена обработка исключения с выводом информативного сообщения пользователю.

В результате проведенной отладки приложение работает корректно, все выявленные ошибки устранены, аварийные завершения программы не происходят.

3.2.6. Осуществление разработки тестовых наборов и тестовых сценариев для программного обеспечения

Тестирование программного обеспечения является неотъемлемой частью процесса разработки, направленной на проверку соответствия программы предъявляемым требованиям и выявление ошибок.

1. Процесс проведения тестирования

Процесс тестирования разработанного приложения включал следующие этапы:

Модульное тестирование (Unit Testing). Данный вид тестирования применялся для проверки корректности работы отдельных изолированных компонентов системы. Основной фокус был сделан на тестировании бизнес-логики, а именно алгоритма расчета скидки, так как это ключевое функциональное требование. Для проведения модульного тестирования использовался фреймворк NUnit, который был установлен в отдельный тестовый проект SadiullinaLibrary.Tests через менеджер пакетов NuGet.

Процесс модульного тестирования включал:

- Создание тестового класса DiscountCalculatorTests, атрибут [TestFixture] указывает, что класс содержит тесты.
- Создание метода инициализации Setup с атрибутом [SetUp], который выполняется перед каждым тестом и создает экземпляр тестируемого класса.
- Написание тестовых методов с атрибутом [Test]. Каждый тестовый метод проверяет один конкретный сценарий.

Использование паттерна AAA (Arrange-Act-Assert) для структурирования тестов:

- Arrange (Подготовка): создание необходимых объектов и подготовка входных данных.

- Act (Действие): выполнение тестируемого метода.
- Assert (Проверка): сравнение полученного результата с ожидаемым с помощью методов класса Assert.

Интеграционное тестирование. Проверялось взаимодействие между различными модулями системы: работа репозиториев с реальной базой данных, взаимодействие сервисов с репозиториями, корректность передачи данных между слоями приложения.

Приемочное тестирование. Выполнялась проверка соответствия реализованного функционала требованиям технического задания. Проверялись все сценарии использования: добавление партнера, редактирование, удаление, просмотр списка, просмотр истории продаж, расчет скидки.

2. Создание unit-тестов

Для тестирования алгоритма расчета скидки был разработан следующий набор модульных тестов:

```
using NUnit.Framework;
using SadiullinaLibrary.Services;

namespace SadiullinaLibrary.Tests
{
    [TestFixture]
    public class DiscountCalculatorTests
    {
        private DiscountCalculator _calculator;

        [SetUp]
        public void Setup()
        {
            _calculator = new DiscountCalculator();
        }

        [Test]
        public void CalculateDiscount_TotalLessThan10000_Returns0()
        {
            // Arrange
            decimal total = 5000;

            // Act
            int discount = _calculator.CalculateDiscount(total);

            // Assert
            Assert.AreEqual(0, discount);
        }

        [Test]
        public void CalculateDiscount_TotalBetween10000And49999_Returns5()
        {
            // Arrange
            decimal total = 30000;
```

```
            // Act
            int discount = _calculator.CalculateDiscount(total);

            // Assert
            Assert.AreEqual(5, discount);
        }

        [Test]
        public void CalculateDiscount_TotalEquals10000_Returns5() // Граничное
значение
        {
            // Arrange
            decimal total = 10000;

            // Act
            int discount = _calculator.CalculateDiscount(total);

            // Assert
            Assert.AreEqual(5, discount);
        }

        [Test]
        public void CalculateDiscount_TotalBetween50000And299999_Returns10()
        {
            // Arrange
            decimal total = 150000;

            // Act
            int discount = _calculator.CalculateDiscount(total);

            // Assert
            Assert.AreEqual(10, discount);
        }

        [Test]
        public void CalculateDiscount_TotalEquals50000_Returns10() // Граничное
значение
        {
            // Arrange
            decimal total = 50000;

            // Act
            int discount = _calculator.CalculateDiscount(total);

            // Assert
            Assert.AreEqual(10, discount);
        }

        [Test]
        public void CalculateDiscount_TotalBetween300000AndAbove_Returns15()
        {
            // Arrange
            decimal total = 350000;

            // Act
            int discount = _calculator.CalculateDiscount(total);

            // Assert
            Assert.AreEqual(15, discount);
        }

        [Test]
        public void CalculateDiscount_TotalEquals300000_Returns15() // Граничное
значение
        {
            // Arrange
            decimal total = 300000;

            // Act
            int discount = _calculator.CalculateDiscount(total);
```

```
                // Assert
                Assert.AreEqual(15, discount);
            }
        }
    }
```

Каждый тестовый метод проверяет определенный диапазон значений, включая граничные условия, что позволяет убедиться в корректности работы алгоритма во всех возможных ситуациях. Запуск тестов осуществлялся через встроенное окно Test Explorer в Visual Studio. Все тесты были успешно пройдены, что подтвердило правильность реализации алгоритма расчета скидки (Рисунок 21).

```
using Moq;
using SadiullinaLibrary.Data;
using SadiullinaLibrary.DTO;
using SadiullinaLibrary.Models;
using SadiullinaLibrary.Services;
using System.Collections.Generic;
using System.Linq;
using System.Threading.Tasks;

namespace Sadiullina.Tests
{
    [TestClass]
    Ссылок: 0
    public class PartnerServiceTests
    {
        private Mock<IPartnerRepository> _mockPartnerRepo;
        private Mock<IProductRepository> _mockProductRepo;
        private Mock<IPartnerTypeRepository> _mockTypeRepo;
        private Mock<IDiscountCalculator> _mockDiscountCalc;
        private PartnerService _service;

        [TestInitialize]
        Ссылок: 0
        public void Setup()
        {
            _mockPartnerRepo = new Mock<IPartnerRepository>();
            _mockProductRepo = new Mock<IProductRepository>();
            _mockTypeRepo = new Mock<IPartnerTypeRepository>();
            _mockDiscountCalc = new Mock<IDiscountCalculator>();

            _service = new PartnerService(
                _mockPartnerRepo.Object,
                _mockProductRepo.Object,
                _mockTypeRepo.Object,
                _mockDiscountCalc.Object
            );
        }

        [TestMethod]
```

Рисунок 21 – Тесты.

3.2.7. Интегрирование программных модулей

Интеграция модулей представляет собой процесс объединения отдельных компонентов системы в единое работающее приложение. В ходе выполнения производственного задания интеграция проводилась по нарастающей

(инкрементальной) схеме, что позволило на каждом этапе получать работающий прототип и своевременно выявлять проблемы взаимодействия между компонентами.

1. Процесс интеграции программных модулей

Процесс интеграции включал следующие этапы:

Интеграция слоя данных. На первом этапе были интегрированы модели данных, контекст базы данных AppDbContext и репозитории. Проверялась возможность подключения к базе данных PostgreSQL, корректность создания таблиц через метод EnsureCreated(), а также выполнение простых запросов через репозитории. Для проверки использовались тестовые запросы на получение данных из созданных таблиц.

Интеграция бизнес-логики. Слой сервисов (PartnerService) был подключен к репозиториям через внедрение зависимостей в конструктор. Проверялась работа методов получения данных с преобразованием моделей в DTO и применением алгоритма расчета скидки. На этом этапе также проверялась корректность обработки исключений, возникающих на уровне доступа к данным.

Интеграция пользовательского интерфейса (ViewModels и Views). На данном этапе осуществлялась привязка представлений к соответствующим ViewModel:

- В конструкторе MainWindow устанавливался DataContext с экземпляром MainViewModel, полученным через внедрение зависимостей.
- Проверялась корректность привязок данных в XAML: отображение коллекции партнеров, обновление интерфейса при изменении выбранного партнера, работа команд.
- Тестировалась навигация между окнами: открытие окна редактирования из главного окна, передача данных между окнами, закрытие окон с возвратом результата.

Настройка DI-контейнера. Финальным шагом стала полная настройка внедрения зависимостей в файле App.xaml.cs. Все необходимые интерфейсы

были зарегистрированы с соответствующими реализациями, настроен контекст базы данных с использованием строки подключения из файла конфигурации appsettings.json.

В результате интеграции все модули (работа с базой данных, бизнес-логика, пользовательский интерфейс) стали работать как единое целое, обмениваясь данными и реагируя на действия пользователя. Была обеспечена корректная передача данных между слоями приложения через объекты DTO, а также согласованная работа всех компонентов системы.

3.2.8. Верификация и аттестация программного обеспечения

Верификация и аттестация являются важнейшими процессами подтверждения качества разработанного программного обеспечения.

Верификация представляет собой процесс проверки того, что программное обеспечение разработано правильно, в соответствии с заданными спецификациями, стандартами и требованиями на каждом этапе жизненного цикла. В ходе разработки верификация проводилась непрерывно и включала следующие мероприятия:

Анализ требований. На начальном этапе была проведена проверка полноты и непротиворечивости понимания требований технического задания. Составлен перечень функциональных требований, подлежащих реализации:

- просмотр списка партнеров;
- добавление нового партнера;
- редактирование данных существующего партнера;
- просмотр истории реализации продукции;
- расчет индивидуальной скидки на основе объема продаж.

Инспекция кода. Проводилась регулярная проверка исходного кода на соответствие стандартам кодирования, правильность реализации алгоритмов, отсутствие логических ошибок и избыточности.

Обзор проектной документации. Проверялось соответствие разработанной архитектуры и базы данных первоначальным проектным

решениям, включая соблюдение требований к нормализации базы данных (3НФ), обеспечение ссылочной целостности, использование согласованной схемы именования объектов

Модульное тестирование. Выполнение модульных тестов подтвердило корректность работы отдельных компонентов, в частности алгоритма расчета скидки.

Аттестация (валидация) представляет собой процесс проверки того, что созданное программное обеспечение соответствует ожиданиям и реальным потребностям заказчика, то есть решает поставленные задачи. Аттестация была проведена путем выполнения приемо-сдаточных тестов, моделирующих реальные сценарии использования системы:

Тест 1: Просмотр списка партнеров.

- Действие: Запуск приложения, открытие главного окна.
- Ожидаемый результат: Отображается список всех партнеров из базы данных в виде карточек с наименованием, ФИО директора, телефоном, рейтингом и рассчитанной скидкой.
- Фактический результат: Список отображается корректно, данные соответствуют записям в базе данных.

Тест 2: Добавление нового партнера.

- Действие: Выбор пункта меню "Партнеры → Добавить партнера", заполнение всех обязательных полей формы, нажатие кнопки "Сохранить".
- Ожидаемый результат: Новый партнер появляется в списке партнеров на главной форме.
- Фактический результат: Новый партнер успешно добавлен и отображается в списке.

Тест 3: Редактирование партнера

- Действие: Выбор партнера в списке, вызов контекстного меню, выбор пункта "Редактировать", изменение некоторых полей, сохранение.
- Ожидаемый результат: Измененные данные отображаются в карточке партнера.

- Фактический результат: Данные обновлены корректно.

Тест 4: Просмотр истории продаж.

- Действие: Выбор партнера в списке.
- Ожидаемый результат: В правой панели отображается история продаж выбранного партнера с указанием наименования продукции, количества и даты продажи. Отображается общая сумма продаж и рассчитанная скидка.
- Фактический результат: История продаж отображается корректно.

Тест 5: Расчет скидки.

- Действие: Выбор партнера с различными объемами продаж, проверка отображаемого значения скидки.
- Ожидаемый результат: Значение скидки соответствует заданной шкале в зависимости от общей суммы продаж.
- Фактический результат: Скидка рассчитывается корректно для всех проверенных значений.

Тест 6: Обработка ошибок.

- Действие: Попытка сохранить партнера с незаполненным обязательным полем.
- Ожидаемый результат: Вывод информативного сообщения об ошибке с пояснением причины.
- Фактический результат: Сообщение об ошибке выводится корректно.

Тест 7: Подтверждение удаления.

- Действие: Выбор партнера, вызов команды удаления
- Ожидаемый результат: Появляется диалоговое окно с запросом подтверждения удаления.
- Фактический результат: Диалоговое окно отображается, при подтверждении партнер удаляется.

Результаты всех тестов положительные. Разработанная подсистема полностью удовлетворяет требованиям технического задания, все функциональные возможности реализованы корректно, пользовательский

интерфейс удобен и соответствует эскизам, обработка исключительных ситуаций обеспечивает информативное взаимодействие с пользователем и предотвращает аварийное завершение программы.

Тема 3.3. Сбор и анализ материалов для оформления отчетной документации по практике

В ходе всей производственной практики осуществлялся систематический сбор и анализ материалов для последующего оформления отчетной документации. Данный процесс включал следующие виды работ:

Ведение дневника практики. Ежедневно фиксировались выполняемые виды работ, их краткое содержание, полученные результаты и возникающие трудности. Дневник велся в соответствии с установленной формой и содержал записи о всех этапах практики: от прохождения инструктажа до завершающих работ по интеграции и тестированию.

Сбор информации о базе практики. Изучались и фиксировались сведения о структуре организации, ее подразделениях, должностных обязанностях сотрудников, используемом программном обеспечении и информационных системах. Анализировались локальные нормативные акты, включая Положение о математическом факультете и должностную инструкцию лаборанта кафедры.

Фиксация процесса разработки. В процессе создания программного продукта сохранялись промежуточные версии кода, делались скриншоты ключевых этапов разработки: диаграммы базы данных, диаграммы классов, интерфейсов окон приложения, результатов тестирования. Собиралась информация об использованных инструментах и технологиях.

Анализ результатов тестирования и отладки. Фиксировались выявленные в процессе отладки ошибки, способы их устранения, результаты выполнения модульных тестов. Это позволило в дальнейшем описать процесс обеспечения качества разработанного программного обеспечения.

Подготовка отчета. На основе собранных материалов был составлен итоговый отчет по производственной практике, включающий:

- введение с указанием цели, задач и места прохождения практики;
- описание организационно-подготовительного этапа;
- характеристику структуры и деятельности подразделения;
- подробное описание процесса разработки программного обеспечения с использованием выбранных методов и инструментов;
- описание процессов инспектирования, отладки, тестирования, интеграции, верификации и аттестации;
- заключение с выводами о результатах практики;
- список использованных источников.

Подготовка презентации и доклада. На основе отчета была подготовлена презентация для защиты результатов практики, включающая основные этапы выполнения индивидуального задания, скриншоты разработанного приложения, диаграммы и ключевые выводы.

Собранные и систематизированные материалы обеспечили возможность составления полного и структурированного отчета, отражающего все этапы и результаты производственной практики.

ЗАКЛЮЧЕНИЕ

В результате прохождения производственной практики ПП.02.01 «Технология разработки программного обеспечения» с 09 по 17 марта 2026 года на базе математического факультета ФГБОУ ВО «Пермский государственный гуманитарно-педагогический университет» (ПГГПУ) были закреплены теоретические знания, полученные в процессе обучения по специальности 09.02.07 «Информационные системы и программирование», и расширены профессиональные умения в области разработки, интеграции и сопровождения программного обеспечения.

По итогам практики получены следующие результаты:

1. Проведен анализ предметной области производственной компании «Мастер пол» и разработана структура базы данных для подсистемы учета партнеров, включающая таблицы типов партнеров, партнеров, продукции, истории продаж и мест продаж. Созданная база данных приведена к третьей нормальной форме, обеспечена ссылочная целостность данных, созданы необходимые первичные и внешние ключи. В соответствии с методическими указаниями все объекты базы данных содержат фамилию студента в своем названии, созданы в схеме app с владельцем app. Получена ER-диаграмма базы данных, сохранен скрипт создания базы данных.

2. Разработано приложение с графическим интерфейсом на платформе .NET с использованием технологий WPF и языка C#. Архитектура приложения построена на основе паттерна MVVM с применением Entity Framework Core для доступа к данным. Приложение разделено на логические слои: модели, слой доступа к данным (репозитории), сервисы (бизнес-логика), DTO, ViewModel и View, что обеспечивает слабую связанность компонентов и упрощает тестирование и сопровождение кода.

3. Реализован полный функционал подсистемы в соответствии с требованиями задания:

- просмотр списка партнеров с отображением в виде стилизованных карточек, содержащих наименование компании, ФИО директора, телефон, рейтинг и рассчитанную индивидуальную скидку;
- добавление нового партнера через отдельное окно с формой, содержащей все необходимые поля (наименование, тип партнера, рейтинг, адрес, ФИО директора, телефон, email);
- редактирование данных существующего партнера с автоматической загрузкой актуальных значений из базы данных;
- просмотр истории реализации продукции партнером с указанием наименования продукции, количества и даты продажи, а также отображением общей суммы продаж и рассчитанной скидки.

4. Разработан и реализован алгоритм расчета индивидуальной скидки для партнера на основании суммарного объема его продаж согласно заданной шкале: до 10 000 руб. – 0%, от 10 000 до 50 000 руб. – 5%, от 50 000 до 300 000 руб. – 10%, более 300 000 руб. – 15%. Алгоритм инкапсулирован в отдельном классе DiscountCalculator, что обеспечивает его независимость и тестируемость.

5. Создана библиотека классов SadiullinaLibrary для размещения моделей, слоя данных и сервисов, что соответствует требованиям модульности. Интерфейс реализован в виде отдельного WPF-приложения Sadiullina. Подключение к базе данных выполнено через Entity Framework Core с использованием роли app для аутентификации.

6. Выполнена интеграция всех модулей в единую систему по нарастающей схеме: интеграция слоя данных, бизнес-логики, пользовательского интерфейса. Обеспечена корректная передача данных между слоями через объекты DTO

7. Проведена отладка программного модуля с использованием специализированных средств Visual Studio 2022 (точки останова, пошаговое выполнение, окна Locals, Watch, Call Stack, Immediate Window). Выявлены и устранены ошибки, включая некорректный расчет скидки при граничных

значениях, проблемы загрузки связанных данных и ошибки валидации. Приложение работает корректно, аварийные завершения не происходят.

8. Разработаны тестовые наборы и выполнено модульное тестирование ключевого алгоритма расчета скидки с использованием фреймворка NUnit. Созданные unit-тесты покрывают все диапазоны значений, включая граничные условия. Все тесты успешно пройдены, что подтверждает корректность реализации алгоритма.

9. Проведено инспектирование компонент программного обеспечения на предмет соответствия стандартам кодирования C# (PascalCase/camelCase именование, форматирование кода, осмысленные идентификаторы, комментирование неочевидных фрагментов). Использовались встроенные средства анализа кода Visual Studio 2022.

10. Разработан единый стилизованный интерфейс пользователя с использованием ресурсов приложения (App.xaml). Созданы стили для основных элементов управления, карточек партнеров, кнопок, текстовых полей. Интерфейс соответствует эскизам, представленным в задании.

11. Реализована обработка исключительных ситуаций с выводом информативных сообщений для пользователя, содержащих полную информацию об ошибках и рекомендации по их устранению. Использованы окна сообщений соответствующих типов (информация, предупреждение, ошибка).

По окончании практики был получен практический опыт:

- интеграции модулей в программное обеспечение;
- отладки программного модуля с использованием специализированных программных средств;
- разработки тестовых наборов и тестовых сценариев для программного обеспечения;
- инспектирования компонент программного обеспечения на предмет соответствия стандартам кодирования;

- проектирования и создания баз данных в третьей нормальной форме с обеспечением ссылочной целостности;
- использования системы контроля версий Git для управления исходным кодом;
- применения паттернов проектирования (MVVM, Repository, Dependency Injection) и принципов SOLID;
- работы с современными инструментами и технологиями разработки (Visual Studio 2022, WPF, C#, Entity Framework Core, PostgreSQL, NUnit).

Все поставленные задачи выполнены в полном объеме, программный продукт успешно прошел верификацию и аттестацию, что подтверждает его соответствие требованиям технического задания. Разработанная подсистема может быть использована для автоматизации учета партнеров и истории продаж в производственной компании.

СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ

1. Федорова Г. Н. Участие в интеграции программных модулей. Учебное пособие / Федорова Г.Н. – М.Ж Издательский дом «Академия», 2016 – 304с.

2. Федорова Г. Н. Разработка, внедрение и адаптация программного обеспечения отраслевой направленности: Учебное пособие. / Федорова Г.Н. - М.:КУРС, НИЦ ИНФРА-М, 2016. - 336 с. http://znanium.com/catalog.php?bookinfo=544732

3. Оценка качества программного обеспечения: Практикум: Учебное пособие / Б.В. Черников, Б.Е. Поклонов; Под ред. Б.В. Черникова - М.: ИД ФОРУМ: НИЦ Инфра-М, 2012. - 400 с. http://znanium.com/bookread2.php?book=315269

4. Гуриков С. Р. Введение в программирование на языке Visual C#: Учебное пособие / С.Р. Гуриков. - М.: Форум: НИЦ ИНФРА-М, 2013. - 448 с. http://znanium.com/bookread2.php?book=232661

5. Исаченко О.В. Программное обеспечение компьютерных сетей : учебное пособие. М. : ИНФРА-М, 2013. - 117 с. - (СПО).

6. Стандартизация, сертификация и управление качеством программного обеспечения: Учебное пособие / Ананьева Т.Н., Новикова Н.Г., Исаев Г.Н. - М.:НИЦ ИНФРА-М, 2016. - 232с http://znanium.com/bookread2.php?book=541003

7. Гагарина Л.Г. Технология разработки программного обеспечения: Учебное пособие / Л.Г. Гагарина, Б.Д. Виснадул, Е.В. Кокорева. М.: ООО "Научно-издательский центр ИНФРА-М", 2013. 400 с. URL: http://znanium.com/catalog.php?bookinfo=389963

8. Немцова Т. И. Программирование на языке высокого уровня. Программир. на языке С++[Электронный ресурс]: Уч. пос. / Т.И.Немцова и др.; Под ред. Л.Г.Гагариной. М.: ИД ФОРУМ: ИНФРА-М, 2012. 512 с. URL: http://znanium.com/catalog.php?bookinfo=244875;

9. Рудаков А.В. Технология разработки программных продуктов: Учеб.пособие для студ. сред. проф. образования. М.: Издательство «Академия», 2010. 208 с.

10. Черников Б.В. Управление качеством программного обеспечения [Электронный ресурс]: Учебник / Б.В. Черников. - М.: ИД ФОРУМ: ИНФРА-М, 2012. - 240 с.. URL: http://znanium.com/catalog.php?bookinfo=256901.

11. Эйдлина Г.М. Delpfi: программирование в примерах и задачах[Электронный ресурс]: Практикум / Г.М. Эйдлина, К.А. Милорадов. М.: ИЦ РИОР: НИЦ Инфра-М, 2012. 116 с. URL: http://znanium.com/catalog.php?bookinfo=319046

Дополнительные источники:

1. Гагарина Л.Г. Разработка и эксплуатация автоматизированных информационных систем [Электронный ресурс]: Учебное пособие / Л.Г. Гагарина. М.: ИД ФОРУМ: НИЦ Инфра-М, 2013.384с. URL: http://znanium.com/catalog.php?bookinfo=368454;

2. Голицына О. Л. Программное обеспечение[Электронный ресурс]: Учебное пособие / О.Л. Голицына, Т.Л. Партыка, И.И. Попов. - 3-е изд., перераб.и доп. М.: Форум, 2010.448с. URL: http://znanium.com/catalog.php?bookinfo=201030;

3. Роббинс, Д. Отладка Windows-приложений [Электронный ресурс] / Д. Роббинс, пер. с англ. М.: ДМК Пресс, 2009. 448 с., ил. URL: http://znanium.com/catalog.php?bookinfo=407747

Интернет-ресурсы:

1. Технология программирования: [Электронныйресурс]. URL: http://glossary.ru/cgi-bin/gl_sch2.cgi?RSl)turujoo!vwujwgssowuigto9

2. Технология программирования:[Электронный ресурс].URL: http://2 programmer.ru/l1?start=3

3. Информатика:[Электронный ресурс]. URL: http:// klgtu.ru/students/literature/inf_asu/600.html

4. Языки программирования: [Электронный ресурс]. URL: http://maksakovsa.ru/TehProgram/index.html

5. Информатика: [Электронный ресурс]. URL: http://object.newmail.ru/obj1.html

6. Delphi и базы данных: [Электронный ресурс]. URL:http://snkey.net/books/delphi/ch4-2.html

7. Базы данных: [Электронный ресурс]. URL: http://www.sdteam.com/5/